

Министерство образования и науки Челябинской области
государственное бюджетное профессиональное образовательное учреждение
«Южно-Уральский государственный колледж»
Кафедры «Информатики и вычислительной техники»

ДОПУЩЕН К ЗАЩИТЕ

Председатель [АИ1]кафедры

«Информатики и вычислительной
техники»

_____ Н. А. Назарова

« ____ » _____ 2026 г.

ДИПЛОМНЫЙ ПРОЕКТ

Разработка игрового приложения на смартфон

специальность СПО 09.02.07 «Информационные системы и
программирование»

Дипломный проект выполнил:

студент группы ИСп450ДК,

очного отделения

_____ Яковлева Я. Д.

Руководитель: преподаватель

_____ Исаев А. Н.

Рецензент: _____

_____ Ф.И.О.

Дата защиты _____

Оценка _____

Челябинск 2026

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

					ПЗ – ЮУГК – 090207 – ДП – 2026		
Изм.	Лист	№ докум.	Подпись	Дата			
Разраб.		Яковлева Я. Д.			Разработка игрового приложения на смартфон		
Провер.		Исаев А. Н.					
Реценз.							
Н. Контр.		Исаев А. Н.					
Утверд.		Назарова Н. А.					
					Лит.	Лист	Листов
					Д	2	51
					ГБПОУ «ЮУГК»		

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	7
1.1 Анализ и общая характеристика предметной области.....	7
1.2 Теоретические аспекты разработки	8
1.3 Анализ средств разработки	12
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....	20
2.1 Проектировка	20
2.2 Реализация проекта	32
2.3 Тестирование.....	42
2.4 Руководство программиста	45
2.5 Руководство пользователя	50
2.6 Экономическое обоснование проекта	57
2.7 Техника безопасности и охрана труда	64
ЗАКЛЮЧЕНИЕ	67
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	70
ПРИЛОЖЕНИЯ.....	74

ВВЕДЕНИЕ

В современном мире мобильные технологии стали неотъемлемой частью повседневной жизни большинства людей. Смартфоны превратились в универсальные устройства, объединяющие функции коммуникации, развлечений, обучения и работы.

На фоне стремительного развития мобильных технологий особую популярность приобрели мобильные игры, которые занимают значительную долю рынка цифровых развлечений. Их доступность и возможность использования в любое удобное время делают их привлекательными для широкой аудитории пользователей всех возрастов. В этих условиях создание качественных игровых приложений становится не только востребованной задачей, но и важным направлением развития современной индустрии разработки программного обеспечения.

Актуальность: разработка мобильных игр представляет собой одно из наиболее перспективных направлений современной индустрии игр за счет повсеместной доступности и распространенности смартфонов. Данная область тесно связана с такими дисциплинами как программирование, дизайн, психология и маркетинг. Также, мобильные игры находят применение в различных сферах: образовании, бизнесе, здравоохранении и социальной коммуникации.

Актуальность разработки мобильных игр заключается в следующем:

1. Формирование цифровой культуры: мобильные игры являются доступным средством развития цифровых навыков у пользователей всех возрастов, способствуют формированию нового уровня взаимодействия с технологиями и информацией.
2. Практическое значение: современные мобильные приложения-игры применяются в образовательных целях, терапевтических практиках и даже в профессиональной подготовке специалистов различных областей.

3. Связь с другими технологиями: разработка мобильных игр стимулирует развитие смежных технологий — от процессоров и графических ускорителей до систем машинного обучения и искусственного интеллекта.

4. Социальная составляющая: мобильные игры становятся платформой для социальных взаимодействий, формирования сообществ и развития новых форм коммуникации между пользователями по всему миру.

В условиях растущей цифровизации общества и увеличения роли мобильных технологий в повседневной жизни разработка качественных игровых приложений приобретает особую значимость как с точки зрения научно-технического прогресса, так и с позиции удовлетворения социально-психологических потребностей современного человека.

Объект исследования: процесс разработки игрового приложения на смартфон.

Цель дипломного проекта: разработать игровое приложение на смартфон.

Задачи дипломного проекта:

- выполнить анализ предметной области разработки мобильных игр;
- изучить теоретические аспекты проектирования и создания игровых приложений для смартфонов;
- проанализировать средства разработки и выбрать оптимальный игровой движок, среду программирования и графический редактор;
- выполнить планирование и проектировку приложения, разработать концепт-документ и техническое задание;
- реализовать программный код, визуальное оформление и функциональные механики игрового приложения в выбранной среде разработки;
- провести функциональное тестирование готового продукта;
- разработать руководство программиста;
- составить руководство пользователя;

- выполнить экономическое обоснование проекта, рассчитав себестоимость разработки и эффективность внедрения;
- рассмотреть вопросы техники безопасности и охраны труда при разработке программного обеспечения.

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
Изм.	Лист	№ докум.	Подпись	Дата		6

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ и общая характеристика предметной области

1.1.1 Общие сведения

Мобильная игра — это развлекательный цифровой продукт, разработанный для использования на смартфонах или других портативных устройствах. Данные приложения спроектированы для управления тач-скрином, оптимизированы под портативные устройства и направлены на короткие игровые сессии.

Первой мобильной игрой, продемонстрировавшей интерес к мобильному геймингу, стала «Змейка», простота и кратковременность игрового процесса которой заложили фундамент для дальнейшего развития мобильных игр, определив ключевые принципы их разработки.

Анализ научных источников позволяет определить, что в настоящее время мобильные игры уверенно занимают большую часть рынка видеоигр. На рисунке 1 показано распределение рынка видеоигр по платформам на 2025 год [22][ЯЯ2].

Рынок видеоигр на 2025 год

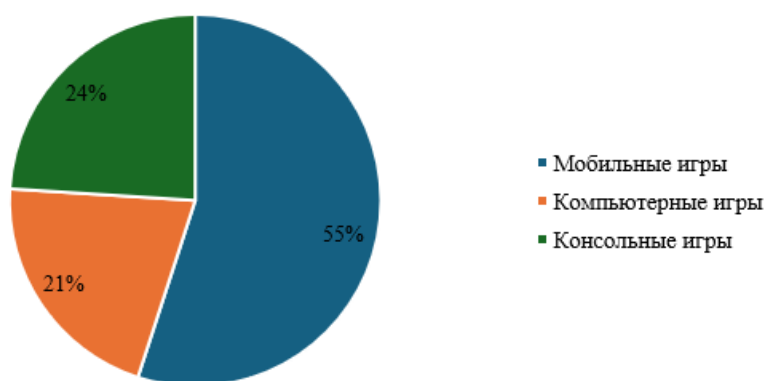


Рисунок 1 — Диаграмма рынка видеоигр на 2025 год

Индустрия растет, развивается и привлекает все больше новых игроков. Пользователю нет необходимости специально выделять время и пространство для того, чтобы провести время в игре — смартфон всегда находится под рукой и интегрирован в повседневную жизнь, что позволяет в любое свободное время

превратить в короткую игровую сессию [18][ЯЯЗ]. Успешность мобильных видеоигр обусловлена особенностью их удобства, доступности и простоты.

1.1.2 Примеры успешных решений

Определение оптимальных проектных решений базируется на анализе опыта популярных проектов в сфере мобильных игр. Анализ помогает выявить сильные и слабые стороны и универсальные решения, которые помогут при проектировке приложения.

Примерами успешных мобильных игр могут служить Candy Crush Saga, Township, Clash Royale. Ниже представлен сравнительный анализ популярных мобильных игр.

Candy Crush Saga — казуальная головоломка, где игроки решают задачи, комбинируя цветные конфеты на игровом поле. Игра предлагает сотни уровней с разнообразными механиками и вызовами.

Township — гибрид симулятора города и фермы, где игроки строят город, управляют производством ресурсов и развивают экономику. Игра сочетает элементы стратегии и управления.

Clash Royale — мультиплеерная карточная стратегия в реальном времени, где игроки собирают карты с персонажами и заклинаниями, чтобы побеждать противников на арене.

Полное сравнение приведенных решений в таблице А.1 приложения А.

Исходя из сравнительного анализа удачным решением будет разрабатывать казуальную игру, с простыми и популярными механиками, запоминающимся дизайном, короткими игровыми сессиями и маленьким порогом входа.

1.2 Теоретические аспекты разработки

Основными аспектами, на которые необходимо опираться и учитывать при разработке мобильной игры, являются: целевая аудитория, технические особенности и способ монетизации [19][ЯЯ4].

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

1. Целевая аудитория

Однозначно описать целевую аудиторию мобильных игр невозможно, так как это большой рынок с множеством различных предложений, однако имеются общие черты всех игроков, по которым можно разделить их на сегменты [22].[ЯЯ5]

Ключевые сегменты целевой аудитории мобильных игр:

- возрастные группы;
- пол;
- географическое положение;
- уровень дохода;
- интересы и хобби.

2. Технические особенности

Мобильная разработка имеет множество условий, которые необходимо соблюдать для того, чтобы итоговый продукт был рабочим. Данные условия происходят из характеристик мобильных устройств, ограничениями их аппаратной части и спецификой взаимодействия с ними.

Платформы смартфонов

Главными платформами — операционными системами, для смартфонов являются Android и iOS. Имеют существенные различия в архитектуре, экосистеме приложения, разработке и пользовательском опыте. Результаты сравнения мобильных платформ по основным характеристикам представлены в таблице 1.

Таблица 1 — Сравнение характеристик мобильных платформ

Характеристика	Android	iOS
Разработчик	Google	Apple
Открытость	Открытая система	Закрытая система
Устройства	Любые	Устройства Apple
Распространение приложений	Google Play Store – основной магазин. Возможна установка из сторонних источников	App Store – единственный официальный способ установки приложений

Выбор платформы для разработки зависит от целей и желания разработчика. Android предоставляет больше свободы и гибкости, но сталкивается с проблемами фрагментации и безопасности. iOS, напротив, предлагает стабильность, безопасность и высокую степень оптимизации, но ограничивает пользователей и разработчиков строгими правилами [18][ЯЯ6].

3. Монетизация

Важной частью разработки приложения, а в частности игры, является выбор стратегии монетизации. Монетизация позволяет разработчику превратить свой продукт в стабильный источник дохода, что позволяет улучшать существующие (продукты) или создавать новые. Наиболее распространенные модели монетизации приведены в таблице 2.

Существует несколько моделей монетизации игр и их использование зависит от жанра, целевой аудитории, конкурентной среды и игрового процесса.

Таблица 2 — Модели монетизации

Модель	Целевые жанры	Ср. доход на пользователя (руб.)
FtP с внутриигровыми покупками	RPG, стратегии, симуляторы	270–1800
Реклама	Гиперказуальные, казуальные игры, головоломки	45–90
Подписки	Кроссжанровая модель, стратегии, экшен-игры	270–1350
Косметика	Шутеры, экшен-игры, ММО, спортивные игры	630–2700
Премиум-модель	Кроссжанровая модель, нишевые игры, головоломки, приключения, инди-игры	450–900

Наиболее популярные подходы являются free-to-play, реклама и продажа косметики [14]. [ЯЯ7] Успех монетизации зависит от того, насколько органично она интегрирована в игровой процесс и не вызывает раздражения у игроков.

4. Дизайн

Главной целью мобильного дизайна является удобство и эффективности взаимодействия при минимальных действиях. Эта цель достигается за счет минимализма, четкой иерархии и интуитивности элементов.

Расположение и размеры

Элементы интерфейса должны быть достаточно крупными и контрастными, чтобы их было удобно использовать на небольших экранах, а их расположение зависит от областей зон взаимодействия с экраном [16].[ЯЯ8]

Область взаимодействия с экраном и расположенными на ней элементами будет обусловлена следующим факторами: ведущая рука пользователя (правша/левша) и ориентация игры. Примеры разделения приведены на рисунках 2 и 3.

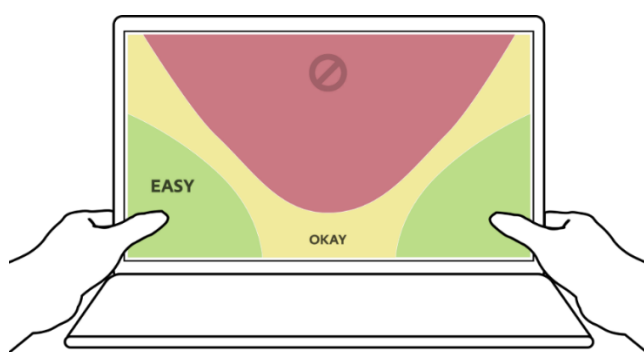


Рисунок 2 — Зоны доступа горизонтальной ориентации



Рисунок 3 — Зоны доступа вертикальной ориентации

Стиль

Цветовая палитра и визуал по большей части определяется требованиями жанра и сеттинга, однако из-за технических особенностей на разработчиков накладываются значительные ограничения на стиль и дизайн игр, что требует от них особого подхода к визуальному оформлению.

Для достижения комфортного дизайна разработчикам необходимо соблюдать правила контрастности, размеров, расположения и читаемости [16][ЯЯ9]. Соблюдение правил позволяет разрабатывать запоминающиеся и

необычные дизайны, сохраняя при этом удобство пользования для игроков, что способствует удержанию пользователей.

Ограничения производительности

Ограниченная производительность смартфонов диктует необходимость оптимизации графики — замена сложных элементов на минималистичные решения, такие как low-poly модели, простые эффекты и компактные ассеты. Все эти факторы формируют общий стиль игры, который стремится к сочетанию функциональности, эстетической привлекательности и технической реализуемости.

Из приведенного выше анализа предметной области мобильных игр и мобильной разработки можно выделить следующие ключевые особенности и характеристики мобильной игры:

- низкий порог входа;
- короткие сессии;
- интуитивное управление;
- быстрый доступ;
- удобство дизайна;
- фокус free-to-play, микротранзакции и рекламу;
- максимальная оптимизация.

1.3 Анализ средств разработки

Разработка игрового приложения задействует множество различных средств и инструментов разработки, ключевыми из которых являются платформа разработки (игровой движок), интегрированная среда разработки (IDE) и инструменты для создания графики. В случае разработки мобильной игры важно учитывать возможность инструментов в игровом движке для разработки на данной платформе.

Анализ средств разработки будет отталкиваться от выбора игрового движка.

1.3.1 Выбор платформы для разработки игр

Платформа для разработки игр (игровой движок) — это набор программного обеспечения, который предоставляет разработчикам игр инструменты и функции для создания и управления игровым контентом.

Для анализа были рассмотрены следующие движки: Unity 2023.1, Unreal Engine 4, Godot.

Unity 2023.1 — кроссплатформенная среда разработки игр, разработанная американской компанией Unity Technologies (рис. 4). Предоставляет широкий набор мощных инструментов для работы с играми, включая оптимизацию и поддержку множества платформ: персональные компьютеры, игровые консоли, мобильные устройства, интернет-приложения и другие [23][ЯЯ10].

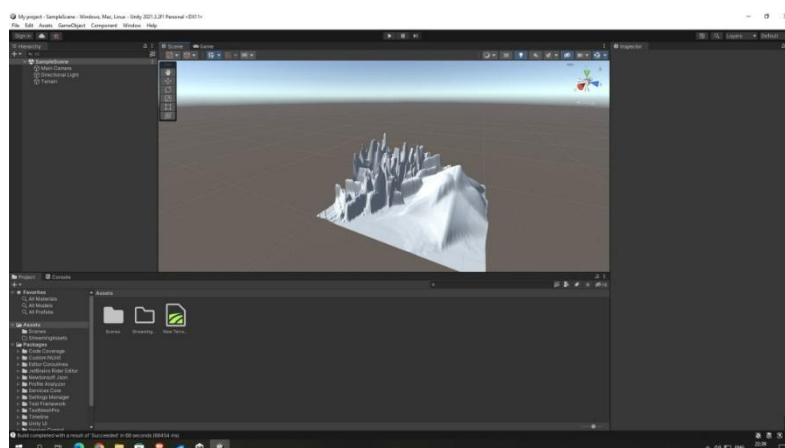


Рисунок 4 — Интерфейс Unity 2023.1

Имеет множество инструментов для работы с 2D-графикой: начиная от системы спрайтов до шейдеров. Для написания скриптов используется язык программирования C#. Среда разработки также имеет инструменты для оптимизации игрового приложения, что позволяет сократить энергопотребление без урона графики и производительности и множество совместимых дополнительных плагинов, позволяющих улучшить и облегчить процесс разработки.

Unity 2023.1 обладает невысокими системными требованиями:

1. ОС: Windows 10 и выше;
2. Процессор: любой 64-разрядный;

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
						13
Изм.	Лист	№ докум.	Подпись	Дата		

3. Видеокарта: поддержка DX от 10;
4. Оперативная память: 8–16 ГБ.

Unreal Engine 4 — игровой движок, разрабатываемый и поддерживаемый компанией Epic Games [26]. Широко используется для крупных проектов на ПК, консолях, мобильных устройствах и в сфере AR/VR. Отличается высокой производительностью, гибкостью и богатым набором инструментов (рис. 5).

Для написания кода использует язык программирования C++, также систему визуального программирования Blueprint. Имеет множество инструментов для настройки графики, функции предпросмотра и эмуляции игры в режиме смартфона, настройки уровней качества графики.

Для оптимизации используются инструменты легковесного рендеринга, сжатия ассетов, методы уменьшения энергопотребления, также стандартные LOD и Occlusion Culling.

Системные требования UE4:

1. ОС: Windows 10 64-разрядная или Windows 7 64-битная;
2. Процессор: четырёхъядерный Intel или AMD с тактовой частотой 2,5 ГГц или выше;
3. Оперативная память: 8 ГБ;
4. Видеокарта: с поддержкой DirectX 11 или DirectX 12.

Godot Engine — открытый кроссплатформенный двухмерный и трёхмерный игровой движок под лицензией MIT, который разрабатывается сообществом Godot Engine Community (рис. 6) [21][ЯЯ11].

Среда разработки запускается на Android, HTML5, Linux, macOS, Windows, BSD и Haiku и может экспортировать игровые проекты на ПК, консоли, мобильные и веб-платформы. Для написания кода используется собственный язык GDScript, чей синтаксис напоминает язык Python. Есть так же возможность программировать на C++, D, Rust, C#.

Системные требования Godot:

1. ОС: Windows 7 SP1+, macOS 10.11+, Linux;

2. Процессор: 64-битный двухъядерный процессор, тактовая частота 2.0 ГГц+;
3. Оперативная память: 4 ГБ+;
4. Графический процессор: с поддержкой OpenGL ES 3.0.

Подробное сравнение игровых движков приведено в таблице А.2 в приложении А.

1.3.2 Выбор интегрированной системы разработки

Для разработки программного кода в выбранном движке Unity 2023.1 могут использоваться Microsoft Visual Studio 2022, Visual Studio Code и JetBrains Rider.

Microsoft Visual Studio 2022 — это интегрированная среда разработки (IDE), предназначенная для разработки программного обеспечения на различных языках программирования, таких как C#, VB.NET, C++, Python и других [27],[ЯЯ12]

Visual Studio включает в себя редактор исходного кода с возможностью простейшего рефакторинга кода (рис. 5). Имеет бесплатную версию Community, которая сразу включает в себя пакет ресурсов для работы с Unity.

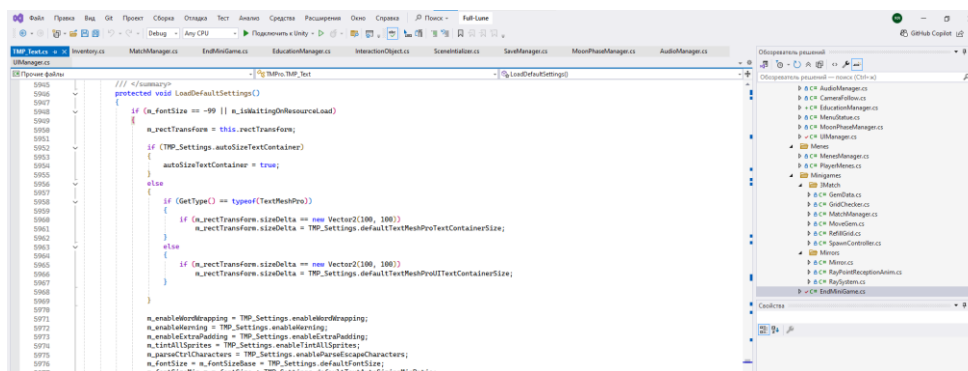


Рисунок 5 — Интерфейс Visual Studio 2022

Visual Studio Code — текстовый редактор, позиционируется как «лёгкий» редактор кода для кроссплатформенной разработки веб- и облачных приложений.

Многие возможности Visual Studio Code недоступны через графический интерфейс, зачастую они используются через палитру команд или JSON-файлы.

Для работы с Unity требуется дополнительная установка расширений и плагинов.

Rider — это платная среда интегрированной разработки от JetBrains, основанная на IntelliJ IDEA. Предлагает широкий набор функций для разработки самых разных приложений с использованием таких фреймворков, как .NET, ASP.NET Core, MAUI, а также игровых движков, например Unity, Unreal Engine или Godot.

1.3.3 Выбор визуального редактора

Важным компонентом в играх составляет визуал. Для разработки визуальной составляющей проекта понадобится графический редактор для рисования.

SAI или **PaintTool SAI** — программа, предназначенная для цифрового рисования в среде Microsoft Windows, разработанная японской компанией SYSTEMAX.

Имеет простой и понятный интерфейс (рис. 6), высокую скорость работы, небольшой вес самой программы, а также все необходимые инструменты для отрисовки спрайтов и работы с пиксель-артом. Имеется поддержка сторонних текстур кистей и текстур холста, которые можно делать собственноручно.

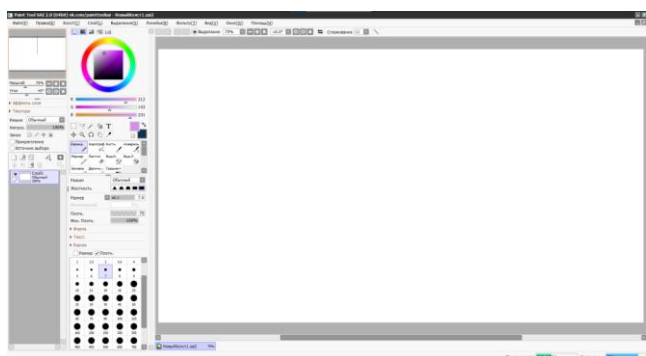


Рисунок 6 — Интерфейс SAI

Adobe Photoshop — профессиональный графический редактор, разработанный компанией Adobe Systems для работы с растровыми изображениями. Широко используется для создания и редактирования изображений, обработки фотографий, дизайна интерфейсов, иллюстраций и других задач визуального контента. Photoshop поддерживает работу с растровой

графикой и имеет множество инструментов для работы с текстурами, слоями, масками и эффектами.

Подробное сравнение графических редакторов приведено в таблице А.3 в приложении А.

В ходе сравнительных анализов игровых движков, сред разработки и визуальных редакторов для разработки были выбраны следующие средства: движок Unity 2023.1, IDE Visual Studio 2022 и графический редактор PaintTool SAI. Обоснованием выбора Unity 2023.1 как игрового движка являются факторы бесплатной лицензии, наличию всех необходимых инструментов для реализации мобильного 2D проекта. Средой разработки выбран Microsoft Visual Studio 2022 за счет беспроблемной поддержки плагинов Unity и полностью бесплатной лицензией. В качестве графического редактора была выбрана программа SAI за счет легкого интерфейса и простоты работы.

1.3.4 Другие инструменты разработки

Качественная разработка проекта требует не только инструментов разработки, но также и сопроводительных инструментов: систему контроля версий, текстового редактора и программу для оформления визуального представления.

Система контроля версий

Система контроля версий (СКВ, GIT) — система, которая регистрирует изменения в одном или нескольких файлах с тем, чтобы в дальнейшем была возможность вернуться к предыдущим его версиям этих файлов [17][ЯЯ13]. Позволяет легко отслеживать и контролировать прогресс разработки, в случае ошибки вернуться и начать с предыдущего шага. Представляет собой набор программ, предоставляющих возможность работы по контролю версий. Это позволяет разрабатывать различные оболочки для повышения удобства работы с программами.

Оболочка GIT — это инструмент, который предоставляет визуальный интерфейс для работы с системой контроля версий, упрощая выполнение

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист 17
Изм.	Лист	№ докум.	Подпись	Дата		

операций. Такие интерфейсы могут быть в виде отдельных приложений, интеграций с IDE или веб-интерфейсов.

Самой популярной GIT-системой является GitHub.

GitHub — самый большой веб-сервис для хостинга проектов и их совместной разработки, часто позиционируется как социальная сеть для разработчиков, предоставляет программу GitHub Desktop — официальный клиент сервиса для облегчения работы с СКВ.

GitHub Desktop напрямую связан с аккаунтом сервиса, а значит связан и с репозиториями пользователя, облегчая связь самого GIT и удаленного репозитория сервиса. Имеет удобный, интуитивный интерфейс (рис. 6). Предоставляет возможность просмотра и работы с внесенными изменений, обозначение имени и комментирование коммита. Также позволяет переключаться между ветками и репозиториями пользователя.

Удобство и легкость в пользовании стали ключевыми факторами в выборе сервиса GitHub для использования.

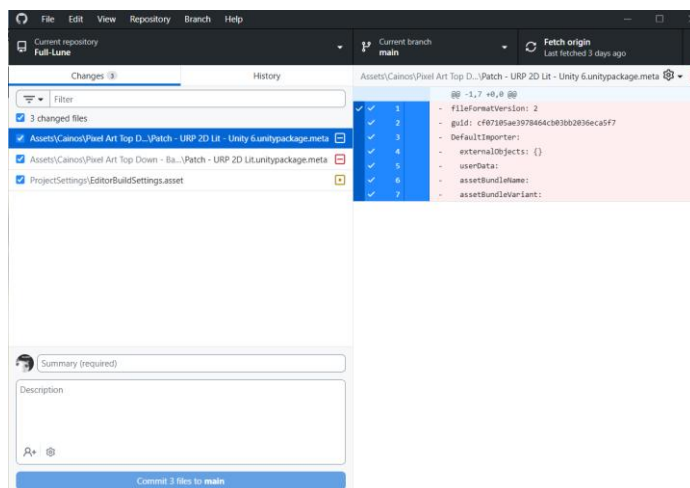


Рисунок 6 — Интерфейс GitHub Desktop

Текстовый редактор

Текстовый редактор — компьютерная программа для создания, редактирования, форматирования и просмотра текстовых данных. Позволяет вводить текст, изменять его, сохранять и открывать файлы.

Наиболее известный текстовый редактор: Microsoft Word.

Microsoft Word — текстовый процессор, разработанный корпорацией Microsoft. Предназначен для создания, редактирования, форматирования и обработки текстовых документов.

Имеет базовые возможности: редактирования текста; работы с изображениями, графиками и таблицами; рецензирования, совместной работы; встроенная функция проверки орфографии.

Программа оформления визуального представления

Программа оформления визуального представления — специализированная программа, предназначенная для структурирования, визуализации информации в виде последовательности слайдов. Позволяет интегрировать текстовые, графические и мультимедийные элементы для наглядной передачи данных.

Наиболее известная программа: Microsoft PowerPoint.

Microsoft PowerPoint — программа для создания и демонстрации презентаций, разработанная корпорацией Microsoft. Предназначен для разработки слайдов, содержащих текст, изображения, диаграммы, видео и анимационные эффекты, с целью визуального сопровождения выступлений или самостоятельного изучения материала.

Имеет базовые возможности: создания и форматирования слайдов с использованием готовых шаблонов и тем; работы с мультимедийным контентом; настройки анимации объектов и переходов между слайдами; экспорта презентации в различные форматы; совместной работы и рецензирования.

Дополнительными средствами разработки были выбраны: GitHub, Microsoft Word и PowerPoint. Выбранные программы удобны в использовании, их функционал достаточен для достижения поставленной цели и задач.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Проектировка

Планирование и проектирование приложения, его структуры и определение требований к нему является одной из важнейших частей разработки программы. На этом этапе формируется представление о готовом продукте, его работе и требованиях к нему. Важно четко определить функции и границы приложения для корректной разработки в дальнейшем [10][ЯЯ14].

В процессе проектирования требуется разработка детального концепт-документа, содержащего ключевые характеристики проекта. В документе должны быть описаны основные параметры: жанр, художественный и нарративный сеттинг, механики, визуальный стиль.

На основе концепт-документа разрабатывается техническое задание, где будет подробно описаны требования к каждому элементу проекта: механикам, процессам, стилям.

Подготовленные при проектировке документы послужат фундаментальной основой для последующей разработки и реализации описанного проекта.

2.1.1 Разработка концепта игры

Концепт-документ мобильной игры «Лунный осколки»

Название: «Лунные осколки»

Жанр: казуальная, головоломки

Сеттинг: мистические земли, деревни, святилища, заброшенные храмы, магия, ритуалы, таинственность места

Платформа: Android, iOS

Возрастное ограничение: 6+

Целевая аудитория

Данные для определения целевой аудитории взяты из бесплатных статей портала Newzoo [22], результаты представлены в таблице 3.

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
Изм.	Лист	№ докум.	Подпись	Дата		20

Таблица 3 — Данные целевой аудитории проекта «Лунные осколки»

Сегмент	Возраст	Пол	География	Уровень дохода
Основной	18–40 лет	Женщины (60–70%)	Россия и СНГ, Балтийские страны, Европа	средний
Второстепенный	12–17 лет	Мужчины (30–40%)	США и страны Азии	низкий

Механики

Цель игры заключается в коллекционировании предметов за счет прохождения головоломок и покупок. Пример игрового цикла приведен на рисунке 7.

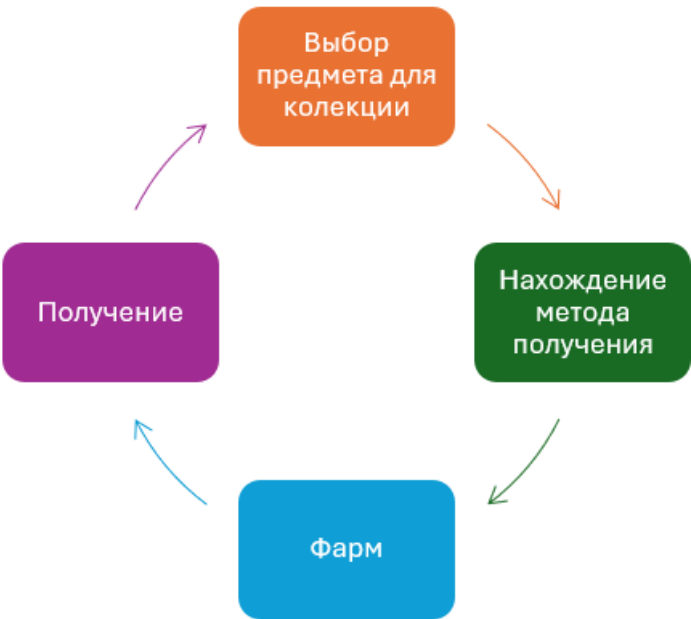


Рисунок 7 — Игровой цикл «Лунных осколков»

1. Лунные осколки

Лунные осколки (далее «ЛО») — внутриигровая валюта, получаемая игроком в качестве награды за прохождение головоломок.

Осколки можно потратить в Лавке, купив коллекционные предметы или смену фазы.

2. Головоломки

За головоломку игроку дается некоторое количество Лунных осколков в качестве награды.

— головоломка «Зеркальца»

Игрок должен повернуть зеркала, чтобы лунный луч попал на алтарь. Каждое зеркало поворачивается только в определённом направлении. После попадания луча на алтарь уровень считается пройденным.

— мини-игра «Камушки-Кумушки»

Игрок должен передвигать камни так, чтобы собрать 3 одинаковых в один ряд (горизонтальный или вертикальный), с целью достигнуть определённых целей уровня.

3. Предметы

Покупаются за ЛО в магазине, хранятся в инвентаре. Дают усиления наград за головоломки. Содержат в своем описании лор игры.

4. Инвентарь

Хранит в себе купленные игроком предметы. Дает возможность посмотреть предметы, имеющимися у игрока, а также их характеристики и описание.

5. Магазин

Предоставляет и хранит список предметов, которые можно хранить, а также те предметы, которые были куплены. Можно просматривать купленные предметы, не купленные предметы и их характеристики.

6. Передвижение персонажа: по сцене и между сцен;

7. Смена фаз и состояний персонажа:

— Новолунье (Дева) — быстрая, скорость передвижения повышенная;

— Полнолунье (Мать) — полная сил, средняя скорость передвижения;

— Старая Луна (Старуха) — ослабленная, пониженная скорость передвижения.

Фазы сменяются самостоятельно каждый 15 минут реального времени. При смене фазы меняется задний фон и индикатор сверху экрана.

Дизайн

Стиль: свободный пиксель-арт

Палитра: палитры для сцен игры обладают спокойными, неяркими, но контрастными цветами.

Локации

1. Деревня Ильмала — главная локация, небольшая деревня с домами жителей и торговыми шатрами.

Архитектура и окружение:

- дома из необожжённого кирпича и дерева;
- центральная площадь со статуей — Статуя Лунной Богини;
- слева проход к Храму первого света, сверху — к Святилищу Неба.

2. Святилище Неба — главный храм в округе, где почитают Менес и Кернунноса.

Архитектура и окружение:

- двухчастное здание: восточный двор — солнечная сторона и западный зал — лунная сторона;
- алтарь Священных камней — активирует мини-игру «Камушки-Кумушки».

3. Храм первого света — храм Менес, заброшенный с момента ее исчезновения, сохранивший историю и атмосферу.

Архитектура и окружение:

- зал, в конце которого — Зеркальный алтарь;
- колонны с трещинами, частью обрушены.

Персонажи

Главный герой: Менес — главная героиня, за которую игрок проходит игру. Потерявшая силы Богиня Луны. Собирает Лунные осколки и предметы для восстановления.

Полный концепт-документ представлен в приложении Б.

2.1.2 Техническое задание

Функциональные требования

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
						23
Изм.	Лист	№ докум.	Подпись	Дата		

Механики

1. Смена фаз: должна происходить автоматическая смена фаз Луны, отображающая индикатором на постоянном игровом интерфейсе, смена настроек для текущей фазы, смена аспекта персонажа.

2. Смена аспекта: должна происходить автоматическая смена аспекта (Дева, Мать, Старуха) для персонажа, смена скорости, удачи. Сменяется вместе с фазой.

3. Лавка предметов: на главной сцене находится ларек, где пользователь может купить коллекционные предметы за Лунные осколки. Каждый предмет можно купить 1 раз, после чего он перестает быть активным в лавке и отображается в инвентаре игрока.

4. Коллекционные предметы: игрок собирает в свою коллекцию предметы путем покупок. Предметы отображаются в инвентаре в меню Статуи.

Головоломки

1. Головоломка «Зеркальца»: игрок должен поворачивать зеркала путем нажатия на них для того, чтобы довести луч из начальной точки до конечной. По завершению должно высвечиваться уведомление о прохождении уровня и начисляться ЛО.

2. Мини-игра «Камушки-Кумушки»: игрок должен передвигать камни, расположенные на игровом поле в случайном порядке таким образом, чтобы 3 одинаковых камня встали в ряд, после чего они должны исчезать, на их месте появляться новые. За каждый камень в составленном ряду начисляется 10 очков. Процедуру необходимо повторять до тех пор, пока не выполнится цель уровня.

Цель уровня генерируется случайно каждый раз в определенном диапазоне, но зависит от текущего аспекта персонажа.

По завершению должно высвечиваться уведомление о прохождении уровня и начисляться ЛО.

Передвижение и взаимодействие

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
Изм.	Лист	№ докум.	Подпись	Дата		24

1. Реализовать перемещение главного персонажа между локациями (Деревня Ильмала, Святилище Неба, Храм первого света).

2. Реализовать интерактивные объекты на локациях: алтари головоломок, знаки переходов, статуя, лавка — и взаимодействие с ними.

Система наград

1. Выдавать Лунные осколки за успешное выполнение головоломок.

2. Учитывать бонус предметов в инвентаре при выдаче ЛО.

Визуальный дизайн

1. Использовать минималистичный пиксель-арт для всех локаций, персонажей и интерактивных объектов.

2. Предписания для визуала локаций:

— Деревня Ильмала: земляной коричневый, охра, серый лён с акцентами белых занавесок и серебряных подвесок;

— Святилище Неба: баланс теплых и холодных тонов;

— Храм Первого Света: серо-голубой, пыльно-белый, тускло-серебряный с акцентами бледно-золотистого и темно-фиолетового.

3. Использовать привлекающие, анимированные знаки для обозначения головоломок.

4. Использовать единый стиль пользовательского интерфейса для кнопок, текста и окон в общем.

Палитра для сцены Деревня Ильмала (рис. 8):

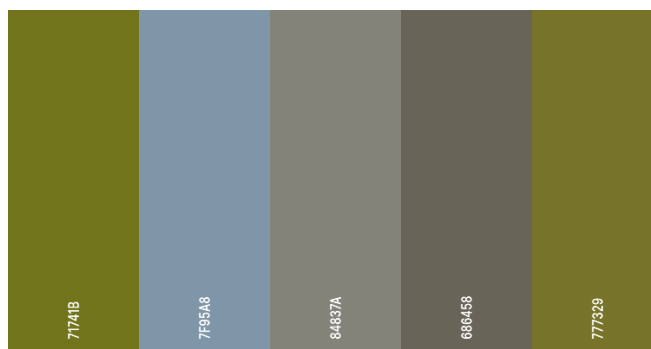


Рисунок 8 — Палитра сцены Деревня Ильмала

Палитра для сцены Деревня Святилища и Храма (рис. 9):

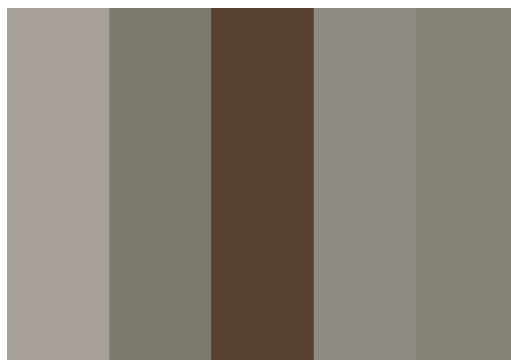


Рисунок 9 — палитра сцены Святилища и Храма

Пример стиля свободного пиксель-арта (рис. 10):



Рисунок 10 — Пример стиля

Дизайн интерфейса

1. Интерфейсные элементы должны быть выполнены в свободном пиксель-арт стиле, но с повышенной контрастностью и четкостью по сравнению с игровым миром. Размер интерактивных зон (кнопок, джойстика) должен соответствовать эргономическим нормам для мобильных устройств, чтобы исключить ложные нажатия при горизонтальной ориентации экрана.

2. Цветовая кодировка функциональных групп UI должна соответствовать установленной палитре для интерфейса (рис. 10).

3. Все окна меню, включая главное меню, обучение и экраны мини-игр, должны использовать идентичную систему закрытия и перехода. Кнопка паузы или выходы из мини-игры всегда располагается в правом верхнем углу (зона большого пальца при горизонтальном удержании).

4. Элементы HUD (индикатор фазы Луны, джойстик, кнопка паузы) должны сохранять фиксированное положение относительно границ экрана независимо от разрешения устройства. При смене фазы визуальный стиль индикатора должен меняться, отражая текущее состояние героини, не требуя от игрока чтения текстовых подсказок.

Палитры для интерфейса (рис. 11):

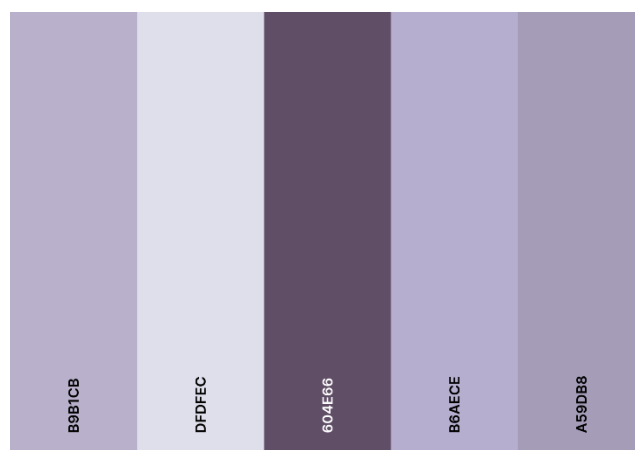


Рисунок 11 — Палитра интерфейса

Макет интерфейсов и пользовательские сценарии

С целью сохранения общего вида дизайна разработан макет (рис. 12), по которому необходимо расположить элементы интерфейса.

Объяснение макета:

1. Цвет сцены соответствует ее типу:
 - фиолетовый — сцены-меню: начальная сцена, сцена обучения;
 - зеленый — свободные зоны: Деремвня Ильмала, Храм Первого Света, Святилище Неба;
 - желтый — сцены головоломки.
2. Стрелки показывают перемещение игрока при нажатии на определенную кнопку интерфейса.
3. Элементы, не имеющие указателей направления (стрелок), предоставляют внутренний функционал сцены (отображение информации, реализация передвижение, показ меню).

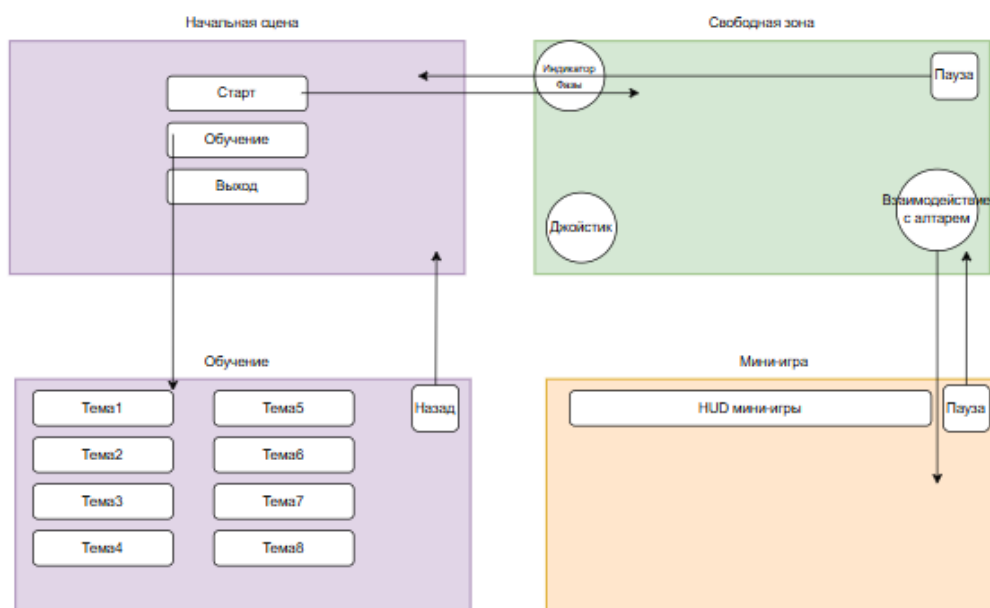


Рисунок 12 — Макет экранов

Анимации

1. Реализовать анимации передвижения персонажа в 4 стороны: вверх, вниз, влево, вправо.
2. Реализовать анимацию бездействия для персонажа.
3. Реализовать анимацию подсказки.
4. Все анимации должны воспроизводиться с частотой не менее 30 FPS на устройствах, соответствующих минимальным техническим требованиям. Целевая частота — 60 FPS.
5. Общий вес всех анимационных ассетов не должен превышать 30 МБ для обеспечения быстрой загрузки приложения на мобильных устройствах.
6. Аниматор не должен застревать в промежуточных состояниях; при потере фокуса или сворачивании приложения все анимации должны корректно останавливаться или сбрасываться в состояние Idle.

Аудио дизайн

1. Для каждой сцены должен быть подобран своя фоновая музыкальная композиция.
2. Композиции должны передавать атмосферу сцены.
3. Композиции не должны быть навязывающими или перетягивающие внимание полностью.

4. Реализовать возможность отключать фоновую музыку в игровом меню игры.

Сохранение игрового процесса[ПЗО15]

1. Игровой процесс должен происходить автоматически при переходах между сценами, выходе из игры.

2. Сохраненные данные должны храниться в формате json-файла.

3. Должны сохраняться следующие данные: аспект персонажа, количество ЛО, фаза Луны, текущее время фазы, инвентарь игрока, состояние Лавки предметов.

Структура приложения[ЯЯ16]

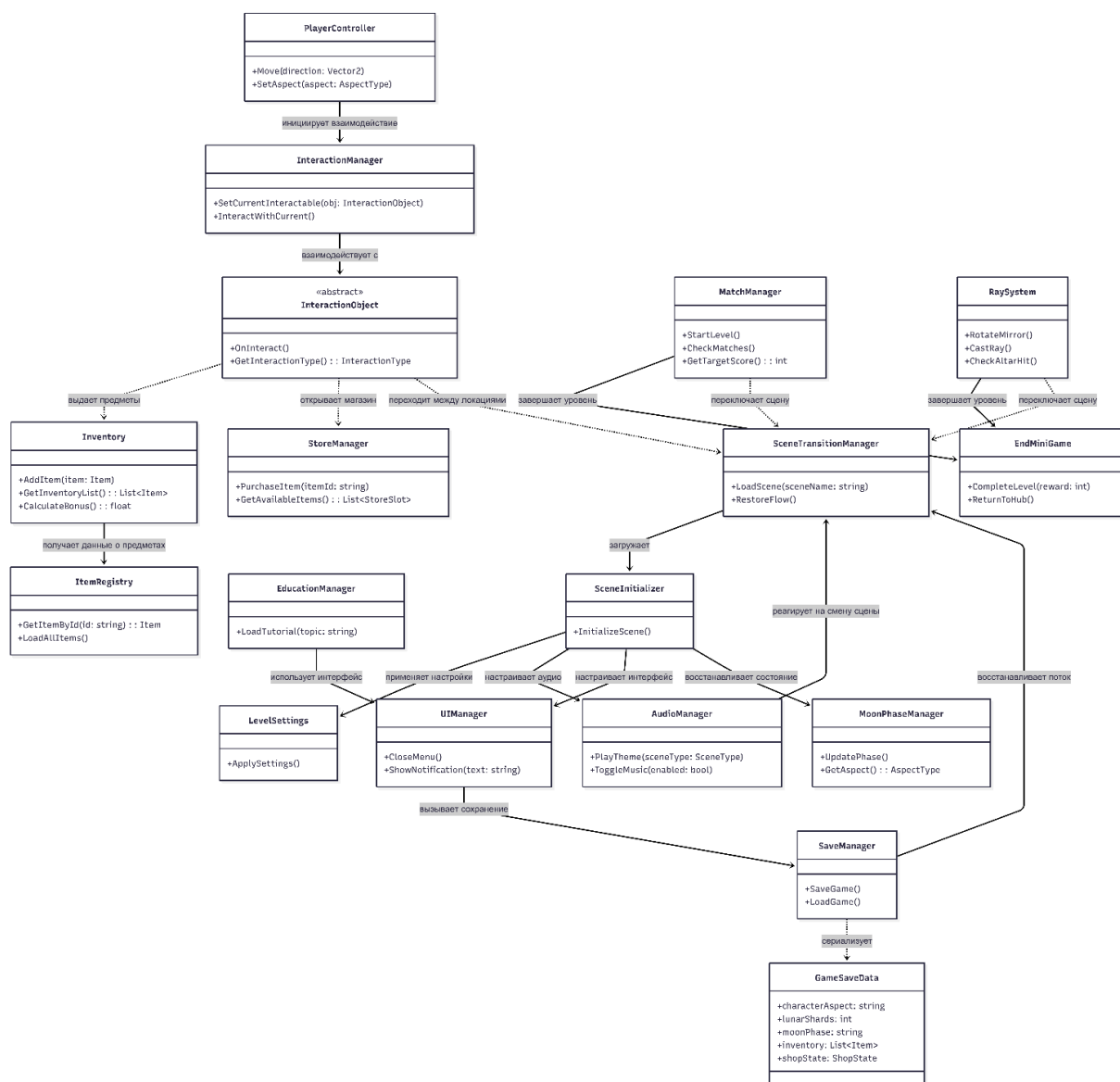


Рисунок 13 — Диаграмма классов

Архитектура приложения должна быть реализована по модульному принципу с использованием паттерна Singleton для глобальных менеджеров и ScriptableObject для хранения конфигурационных данных. Диаграмма классов (рис. 13) отражает ключевые компоненты системы и их взаимодействия.

Система разделена на четыре функциональных подсистемы:

1. Ядро выполнения

Управляет жизненным циклом приложения. Класс SceneTransitionManager обеспечивает бесшовную загрузку сцен и сохранение точки входа. SaveManager отвечает за сохранение данных через сериализацию модели GameSaveData в JSON-формат.

Глобальные синглтоны MoonPhaseManager и AudioManager сохраняют свое состояние между сценами, обеспечивая непрерывность игрового процесса.

2. Исследование мира

Реализует механику взаимодействия игрока с окружением. Абстрактный класс InteractionObject определяет контракт для всех интерактивных элементов (алтари, знаки, статуя), что позволяет системе InteractionManager унифицированно обрабатывать события.

Подсистема предметов использует реестр ItemRegistry для централизованного доступа к данным коллекционных объектов, а StoreManager и Inventory управляют состоянием покупок и инвентаря соответственно.

3. Мини-игры

Представлены двумя независимыми контроллерами: MatchManager (головоломка «Камушки-Кумушки») и RaySystem (головоломка «Зеркальца»). Оба компонента делегируют логику завершения уровня и начисления награды единому классу EndMiniGame, что будет обеспечивать стандартизированный возврат в основной игровой процесс.

4. Представление и управление

Включает UIManager для координации интерфейсных окон и PlayerController для обработки ввода и анимации персонажа. Аудиосистема

(AudioManager) интегрирована с менеджером сцен для автоматической смены тематического трека при переходе между локациями.

Технические требования

Минимальные технические требования:

Таблица 4 — Минимальные технические требования приложения

Компонент	Требование
Операционная система	Android 8.0 (Oreo) или выше
Процессор	Quad-core 1.4 GHz Cortex-A53 или аналогичный
Оперативная память	2 ГБ
Графический процессор	Adreno 405 / Mali-T720 или аналогичный
Место на диске	200–300 МБ свободного места
Разрешение экрана	1280x720

Требования к производительности

Разработка приложения должна обеспечивать стабильную и плавную работу на целевом устройстве Samsung Galaxy S21 FE при работе в режиме энергосбережения с полным зарядом аккумуляторной батареи.

Значения частоты кадров не должны быть ниже минимальных показателей FPS и требований к времени выполнения операции, приведенных в таблицах 5 и 6 соответственно. При несоответствии производительности приложения следует провести поиск и решение проблемы.

Таблица 5 — Целевые показатели частоты кадров

Режим/сцена	Целевой FPS	Минимальный FPS
Главное меню и начальный экран	60	55
Свободные локации (Деревня Ильмала, Святилище Неба, Храм первого света)	60	55
Мини-игра «Зеркала»	60	50
Мини-игра «Камушки-кумушки»	60	45
Сцена обучения	60	55
Переходы между сценами (загрузка)	30	24

Таблица 6 — Временные характеристики

Операция	Требование ко времени
Запуск приложения до главного меню	не более 4 секунд
Загрузка свободной локации (сцены)	не более 2,5 секунд
Загрузка сцены мини-игры	не более 2 секунд
Сохранение игрового прогресса (JSON)	не более 0,3 секунды
Загрузка сохранённого прогресса	не более 1,5 секунды

2.2 Реализация проекта

2.2.1 Разработка визуала

Для разработки визуала использовались бесплатные, свободно распространяемые ассеты, размещенные на платформе Unity Asset Store [], а также те, что были разработаны самостоятельно.

Локации

Локации игры созданы с помощью инструментов Tilemap и Tile Palette, которые позволяют разметить игровое поле в виде сетки, на которой можно рисовать блоками Tile Palette.

Главная локация — деревня Ильмала. В ней находится Статуя Богини, небольшой деревенский рынок, домики жителей и проходы на другие локации в виде указателей направления. Готовая планировка Деревни представлена на рисунке 14.



Рисунок 14 — Деревня Ильмала

Храм первого света (рис. 15) — вторая локация. На локации находится алтарь для активации мини-игры «Зеркала», выход обратно в деревню и декоративные элементы в виде колонн, сундуков и неактивных алтарей.



Рисунок 15 — Храм Первого света

Святылище Неба — третья локация игры. Имеет запуск мини-игры «Три в ряд», выход на локацию деревни, заполнение в виде колонн, скамеек, неактивных алтарей и сундуков, также сделано упрощённое графическое разделение Святылища на сторону Солнца и сторону Луны (рис. 16).

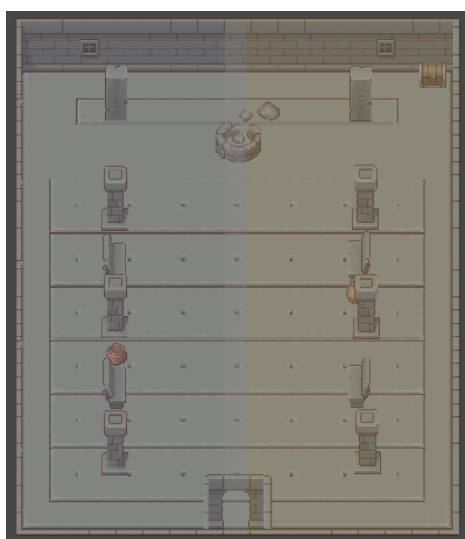


Рисунок 16 — Святылище Неба

Интерфейс

При реализации интерфейса должны быть реализованы следующие элементы: меню паузы, панель информации Статуи, отображение текущей фазы луны, интерфейсы мини-игр, а также главное меню игры. Все элементы должны соблюдать один стиль для поддержания визуального оформления игры.

1. Начальный экран

Представляет собой набор UI-элементов, необходимых для представления и начала игры: название, кнопка старта, кнопка обучения и кнопка выхода из игры.

2. Игровой интерфейс

Сохраняется во время всей игры на сценах свободной зоны. Позволяет пользователю передвигаться, отслеживать игровую фазу и взаимодействовать с меню паузы. На рисунке 17 представлен постоянный игровой интерфейс, который отображается во время всей игровой сессии.



Рисунок 17 — Постоянный интерфейс игры

Имеет 3 элемента:

- левый верхний угол: индикатор фазы;
- правый верхний угол: кнопка паузы и меню паузы;
- левый нижний угол: джойстик управления.

3. Меню паузы

Содержит 3 функциональных кнопки:

- продолжить: закрывает меню, игра продолжается;
- аудио: кнопка-переключатель, управляет включением и выключением фоновой музыки;
- выход: закрывает приложение.

4. Панель Статуи

Панель Статуи — это информационная панель, позволяющая пользователю просматривать игровую информацию: состояния персонажа, текущую фазу, инвентарь сюжетных предметов.

5. Меню паузы для головоломок

Для головоломок отдельно разработано меню паузы. Оно спрашивает у игрока о решение выйти с уровня и предупреждает о потере процесса. Представляет собой панель с текстом и 2 кнопками (Да/Нет).

2.2.2 Создание скриптов

Скрипты персонажа

Игровой персонаж (Менес) управляется двумя компонентами:

PlayerMenes — реализовывает в себе механику передвижения, хранения значения аспекта, хранение модификаторов скорости;

Скрипт-менеджер MenesManager является глобальным менеджером, но относится к управлению персонажем, так как управляет его характеристиками. Сохраняется при переходах.

Код в приложении В.1.

Скрипты-менеджеры

К игровым менеджерам относятся скрипты, которые управляют общим игровым процессом, сценами и загрузкой данных.

Менеджеры разделяются на обще игровые — большая их часть сохраняется между сценами, управляют глобальными процессами игры, менеджеры сцен — управляют процессами на сцене: загрузка сцены, управление сценой и скрипты управления интерфейсом.

К глобальным игровым скриптам относятся:

LevelSettings — отвечает за настройки сцен. Содержит в себе настройки для сцен. Позволяет задать общие настройки в инспекторе 1 раз и в дальнейшем их использовать на любой сцене. Сохраняется при переходах.

SceneTransitionManager — задает настройки переходов между определенными сценами – отслеживает с какой сцены пришел игрок и задает для текущей сцены точку появления. Сохраняется при переходах.

MoonPhaseManager — глобальны скрипт, отвечающий за смену фазы Луны и аспекта персонажа по таймеру. Хранит и сменяет фоны сцен и индикатор фазы на игровом интерфейсе. Сохраняется при переходах.

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист 35
Изм.	Лист	№ докум.	Подпись	Дата		

AudioManager — менеджер фоновой музыки приложения. Хранит в себе настройки различных композиций для сцен. Позволяет включать и выключать музыку на сценах. Сохраняется при переходах.

Код в приложении В.2.

К **скриптам сцен** и другим относятся:

CameraFollow — код, отвечающий за следование камерой игрового персонажа. Позволяет настроить цель следования и скорость передвижения камеры. Прикрепляется на каждой сцене к камере.

SceneInitializer — скрипт, прикрепленный к каждой сцене. Через него задается тип текущей сцены. Вызывает метод установки настроек менеджера LevelSettings.

InteractionManager — менеджер, отвечающий за взаимодействие с объектами. Управляет текущим объектом, с которым взаимодействует игрок, позволяет вызывать метод взаимодействия.

InteractionObject — базовый класс интерактивных объектов. Реализуется дочерними классами объектов (Altar, RoadSing, Statue).

Код скриптов в приложении В.3.

Менеджеры интерфейса:

UIManager — скрипт с общими методами: загрузка первой сцены, выход из игры, закрытие меню, сохранение игрового процесса.

MenuStatue — специальный скрипт для объекта Статуи и ее информационной панели. Содержит в себе метод обновления информации, который собирает данные с разных источников и предоставляет ее пользователю в наглядном виде.

MenuExit — отдельный скрипт для меню мини-игр. Отвечает за выход из головоломки и предупреждение пользователя о сброшенном игровом процессе.

EducationManager — скрипт для сцены обучения. Предоставляет хранение гайдлайнов, их выбор и просмотр. Также содержит класс EducationTopic, хранящий в себе содержание каждой темы обучения.

Код скриптов в приложении В.4.

Скрипты предметов, магазина, инвентаря

Предметы, магазин и инвентарь представляют тесно связанную систему объектов и скриптов, напрямую влияющую друг на друга.

Магазин и инвентарь имеют похожую структуру за счет хранения одних и тех же данных (коллекционных предметов). Такая структура позволяет легко руководить их функционалом и является понятной для пользователя.

Вся система разделена на 3 целевых компонента: предметы, магазин и инвентарь.

Предметы (Item Collection)

Item — специальный тип скрипта (ScriptableObject), позволяющий создавать свои игровые объекты, работать с ними вне сцены и сериализовывать данные. Предоставляет возможность создавать в редакторе коллекционные объекты, назначать им название, спрайт, описание, цену и бонус.

ItemRegistry — скрипт, записывающий в себя все объекты Item и хранящий список до конца игровой сессии. Вызывается при загрузке первой сцены, предоставляет метод поиска предмета в списке по уникальному идентификатору.

Инвентарь (Inventory)

Inventory — менеджер управления инвентарем. Отвечает за запись предмета в пустующий слот, сохранение списка предметов, суммирование бонуса со всех предметов.

InventorySlot — скрипт ячейки инвентаря. Хранит в себе предмет, отображает его спрайт, позволяет вызывать панель информации о хранящемся предмете (скрипт ItemInfo).

ItemInfo — показывает подробную информацию о предмете в всплывающем окне.

Магазина (Store)

StoreManager — менеджер управлению магазином. Хранит в себе список ячеек (StoreSlot), их предметы, состояние ячейки. Отображает подробную

информацию о выбранном предмете, имеет метод покупки. Также отвечает за сохранение данных магазина.

StoreSlot — скрипт ячейки магазина. Хранит в себе предмет, отображает его спрайт, позволяет выбирать его для покупки. Имеет 2 состояния: куплено и не куплено.

Код скриптов в приложении В.5.

Скрипты сохранения

Сохранение игрового процесса реализовано через json-файл, класс-данных и скрипта управления сохранениями.

Класс GameSaveData хранит в себе структуру записываемых в json-файл данных, что позволяет упорядоченно хранить и обращаться к ним.

Скрипт SaveManager отвечает за сохранение и загрузку данных. Является статическим, так как предоставляет набор функций, не требует присутствия на сцене, также это облегчает вызов методов загрузки и сохранения, ведь не нужно создавать экземпляр класса.

В игре сохранение реализовано таким образом, что при переходе на новую сцену вызывается загрузка данных и их сохранение, что позволяет пользователю не беспокоиться постоянно о своем игровом процессе.

Код скриптов в приложении В.6.

2.2.3 Создание анимаций персонажа

Для анимации используются готовые раскадровки из ассета (рис. 18).

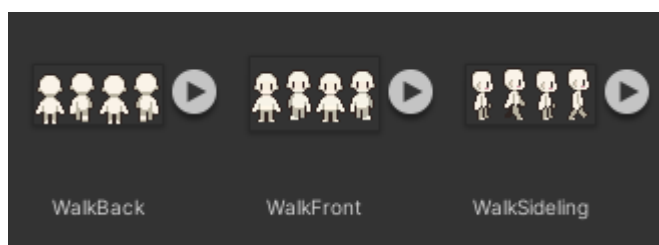


Рисунок 18 — Раскадровки анимации персонажа

IdleFront — анимация бездействия, персонаж лицом к камере, проигрывается всегда, когда персонаж стоит;

WalkBack — анимация передвижения, персонаж спиной к камере, проигрывается при движении вверх;

WalkFront — анимация передвижения, персонаж лицом к камере, проигрывается при движении вниз;

WalkRight — анимация передвижения, персонаж боком к камере, проигрывается при движении вправо;

WalkLeft — анимация передвижения, персонаж боком к камере, проигрывается при движении влево.

2.2.4 Создание головоломок

Головоломка «Зеркала»

Суть головоломки заключается в том, чтобы благодаря зеркалам «доставить» луч из начальной точки до точки назначения.

Для головоломки необходимо иметь 3 префаба: зеркала, точки выпуска луча и точки приема луча (рис. 19).

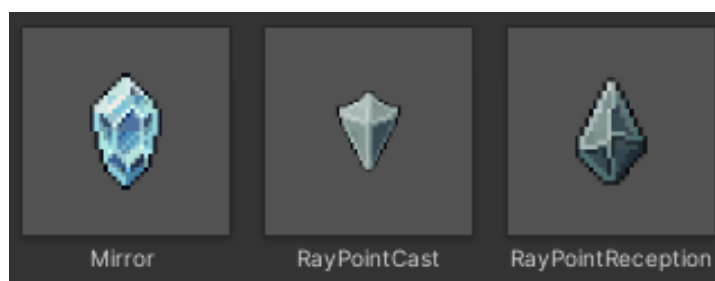


Рисунок 19 — Префабы для головоломки «Зеркала»

1. Зеркала

Имеют коллайдер в виде капсулы, подогнанный максимально под форму предмета для точности отражения луча. Находится на специальном слое «Mirror» для правильного движения луча. При нажатии поворачиваются на 45 градусов вправо за счет скрипта Mirror.

2. Точка создания луча

Имеет компонент Line Render для отображения и отрисовки луча с помощью скрипта RaySystem.

3. Точка приема луча

Имеет Box Collider для обработки столкновений с лучом и скрипт RayPointReceptionAnim для анимации при прохождении уровня.

Скрипты

Скрипт Mirror отвечает за вращение зеркал на сцене и взаимодействием с ними. Размещается на объектах «Зеркал», позволяет обрабатывать нажатие по зеркалу, вызывая метод RotateMirror, который поворачивает зеркало на 45 градусов.

Скрипт RaySystem обрабатывает отрисовку и столкновения луча, отражение от Зеркал и достижение конечной точки, а также начисление Лунных соколов после прохождения уровня.

Вспомогательный RayPointReceptionAnim отвечает за анимацию переливания цвета точки приема по завершению уровня, EndMiniGame вызывает меню, уведомляя игрока о конце уровня, начисляет ЛО и возвращает игрока на уровень, откуда была открыта головоломка.

Код скриптов в приложении Г.1.

Мини-игра «Камушки-кумушки»

Данная мини-игра разработана на основе обычной игры «Три в ряд» и сохраняет в себе базовые механики родителя. Суть заключается в наборе заявленной уровнем цели путем перестановки камней на заранее определенном поле таким образом, чтобы 3 и более одинаковых камня встали в ряд. При совпадении камней они удаляются с поля и начисляются очки, на место удаленных камней появляются новые.

Цель уровня рассчитывается динамически и зависит от текущего аспекта Менес. Изначальное число необходимых очков лежит в диапазоне от 3 до 333, получаемое методом генерации числа из заявленного диапазона.

Далее, в зависимости от текущего аспекта, устанавливается коэффициент модификации числа очков. Значения для модификации для каждого аспекта приведены в таблице 7.

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
						40
Изм.	Лист	№ докум.	Подпись	Дата		

Таблица 7 — Коэффициенты для каждого аспекта

Аспект	Дева	Матерь	Старуха
Коэффициент	1.3	1	0.3

Конечный результат вычисляется по формуле 1:

$$\text{countTarget} = \text{target} + (\text{target} * \text{coeff}) \quad (1)$$

Для реализации потребовались:

1. 4 префаба разных камней[ПЗО17]



Рисунок 20 — Префабы камней для мини-игры

Каждый префаб имеет circle collider 2D, скрипт DemData — общие компоненты для все камней. Различием является уникальный тег каждого камня, по которому на игровом поле главный менеджер игры различает их.

2. Игровое поле

Изначально пустой объект, после начала уровня, который заполняется сеткой камней.

Скрипты

MatchManager — главный менеджер уровня, управляющий игровым полем, отслеживанием процесса, запуском процессов изменения поля, установкой цели уровня.

GemData — скрипт каждого камня, управляющий задающий его координаты на поле.

GridChecker — скрипт проверки поля на наличие совпадений. Предоставляет метод, который проверяет сетку камней по горизонтали и вертикали на совпадение 3 и более камней. Передает в MatchManager число совпадающих объектов, данные состояния поля (возможность заполнения, передвижения, проверки).

MoveGem — скрипт, отвечающий за считывание касания экрана для перемещения камней. Определяет выбранный камень, направление перемещения и передает координаты в GemData для передвижения.

RefillGrid — скрипт заполнения игрового поля по мере игры. Запускает создание новых камней с анимацией падения в свои ячейки.

SpawnController — скрипт, заполняющий игровое поле в начале уровня. Проверяет сетку на начальные совпадения, удаляет их при наличии.

Код скриптов в приложении Г.2.

Для всех мини-игр имеется общий скрипт EndMiniGame, отвечающий за окончание уровня. Он выводит пользователю уведомление об окончании уровня, вычисляет полученные ЛО и начисляет их игроку. Код скрипта в приложении Г.3.

2.3 Тестирование

2.3.1 Функциональное тестирование

Для проверки соответствия игры требованиям необходимо провести тестирование приложения, которое поможет выявить проблемы и найти их решения [1].

Тестирование — это процесс проверки соответствия программы требованиям, спецификации или ожиданиям конечного пользователя. Оно включает в себя запуск программы с целью выявления ошибок, дефектов или недочётов. Тестирование может быть проведено как вручную, так и автоматически с помощью специальных инструментов [20][ЯЯ18].

Оптимальным методом тестирования для данного проекта метод черного ящика: тестирование функционала и производительности. Выбранные виды тестирования дадут достаточный обзор на состояние проекта, его соответствие установленным требованиям и ожиданием пользователей.

Функциональное тестирование

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
Изм.	Лист	№ докум.	Подпись	Дата		42

Тестирование функциональности — тестирование пользовательского интерфейса, ввода/вывода, работы с файлами и т. д., без прямого доступа к исходному коду программы. Тестировщик проверяет, что программа выполняет требуемые действия в соответствии с заданными в спецификации требованиями. [20][ЯЯ19].

Основными направлениями функционала, который подвергается тестированию, будут механики: передвижений, смены фаз, перемещений между локациями, взаимодействием с интерактивными объектами; системы мини-игр, лавки предметов и инвентаря и настроек сессии (игровые менеджеры).

Тестирование проводится на целевом устройстве Samsung S21 FE с полным зарядом батареи, на оптимальных настройках.

В результате тестирования все проверяемые функции отработали в соответствии с ожидаемыми значениями. Полные результаты функционального тестирования приведены в таблице Д.1 в приложении Д.

Тестирование производительности

Тестирование производительности — тестирование производительности программы, измерение времени работы, использования ресурсов, сравнение с заданными требованиями по производительности. [20][ЯЯ20]

Платформа разработки Unity предоставляет специальные инструменты разработчикам для тестирования производительности проекта: профилировщик (Profiler).

Unity Profiler — это инструмент для анализа производительности приложений, разработанных на движке Unity. Позволяет выявлять узкие места, оптимизировать код и ресурсы, а также отслеживать использование ресурсов в реальном времени [28][ЯЯ21].

Профилировщик встроен в редактор Unity и не требует дополнительных установок. Отображает данные о производительности в области рендеринга, аудио и использования памяти (рис. 21).

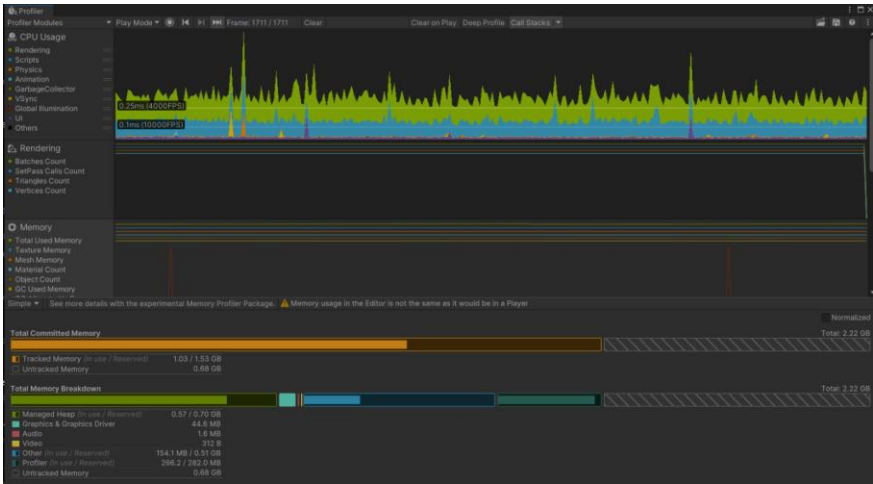


Рисунок 21 — Интерфейс Unity Profiler

Тестирование проводится на целевом устройстве Samsung S21 FE с полным зарядом батареи, на оптимальных настройках.

В ходе тестирования производительности отслеживалось значение ФПС при ключевых взаимодействиях: переход между сценами, передвижение персонажа, открытие интерфейсов, процесс прохождения мини-игр. Тестирование проводилось линейно, по пользовательскому сценарию, охватывая все возможные взаимодействия.

Результаты тестирования производительности представлены на рисунке 22 в виде графика. Точки графика — ключевые моменты: статичное состояние сцен, взаимодействие с объектами, переходы между сценами и значение ФПС в данные момент времени.



Рисунок 22 — Результат тестирования производительности

По представленному результату среднее значение частоты кадров: 157 кадр/с с учетом просадки при переходах между сценами. Целевое значение ФПС при нахождении на сцене и взаимодействия с ней: 256 кадр/с.

Наблюдаемые просадки производительности до 40–70 кадр/с фиксируются исключительно в моменты загрузки и перехода между сценами, что является технически обоснованным, так как в эти моменты происходит:

- инициализация игровых объектов;
- загрузка текстур и ассетов;
- сериализация и десериализация данных сохранения.

Длительность данных просадок не превышает 0,3–0,5 секунды, что находится в пределах допустимых значений для мобильных приложений и не оказывает негативного влияния на пользовательский опыт.

Исходя из результатов, можно сказать, что производительность приложения соответствует установленным требованиям.

Результат тестирования: приложение полностью соответствует поставленным техническим требованиям из технического задания, что означает, что приложение успешно прошло тестирование.

2.4 Руководство программиста

2.4.1 НАЗНАЧЕНИЕ И УСЛОВИЯ ПРИМЕНЕНИЯ ПРОГРАММ

2.4.1.1 Назначение программы

Руководство программиста предназначено для разработчиков, участвующих в создании и сопровождении мобильного игрового приложения «Лунные осколки». Программа представляет собой мобильное приложение для смартфонов, реализующее игровые механики, связанные с фазами Луны, коллекционированием предметов и прохождением мини-игр [12][ЯЯ22].

Основные функции программы:

- реализация игрового процесса с механиками смены фаз Луны;
- управление аспектом персонажа;

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
Изм.	Лист	№ докум.	Подпись	Дата		45

- коллекционирование Лунных осколков (ЛО);
- реализация мини-игр «Зеркала» и «Камушки-Кумушки»;
- управление инвентарем игрока;
- реализация магазина предметов (Лавка);
- сохранение и загрузка игрового прогресса;
- музыкальное сопровождение с возможностью отключения.

2.4.1.2 Условия применения программы

Для корректной работы приложения "Лунные осколки" необходимо соблюдение следующих условий:

Требования к аппаратному обеспечению:

1. Операционная система: Android 8.0 (Oreo) или выше.
2. Процессор: Quad-core 1.4 GHz Cortex-A53 или аналогичный.
3. Оперативная память: 2 ГБ.
4. Графический процессор: Adreno 405 / Mali-T720 или аналогичный.
5. Место на диске: 200–300 МБ свободного места.
6. Разрешение экрана: 1280x720.

Требования к программному окружению:

- среда разработки: Unity 2023.1
- интегрированная среда разработки: Microsoft Visual Studio 2022
- графический редактор для создания ассетов: PaintTool SAI

Программа разработана с использованием языка C# и предназначена для работы на мобильных устройствах под управлением операционной системы Android.

2.4.2 ХАРАКТЕРИСТИКА ПРОГРАММЫ

2.4.2.1 Основные характеристики и особенности программы

Приложение «Лунные осколки» представляет собой 2D-игру с пиксельной графикой, реализованную с использованием игрового движка Unity. Основные особенности программы:

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
						46
Изм.	Лист	№ докум.	Подпись	Дата		

- автоматический игровой процесс при переходах между сценами и выходе из игры;
- сохранение игрового прогресса в формате JSON;
- поддержка нескольких игровых механик: смена фаз Луны, изменение аспекта персонажа;
- наличие коллекционных предметов и мини-игр;
- единый визуальный стиль, соответствующий требованиям мобильного дизайна.

2.4.2.2 Временные характеристики

- скорость загрузки сцен: не более 3 секунд;
- время сохранения игрового прогресса: не более 1 секунды;
- время загрузки сохраненного прогресса: не более 2 секунд;
- стабильная работа при 60 FPS на устройствах, соответствующих минимальным требованиям.

2.4.2.3 Режим работы

Программа работает в следующих режимах:

- режим игры (основной режим);
- режим меню (главное меню, настройки, инвентарь);
- режим мини-игр («Зеркала», «Камушки-Кумушки»);
- режим обучения (обучение игровым механикам).

2.4.2.4 Средства контроля и самовосстанавливаемости

- автоматическое сохранение прогресса каждые 5 минут игрового времени;
- проверка целостности сохраненных данных при загрузке.

2.4.3 ОБРАЩЕНИЕ К ПРОГРАММЕ

2.4.3.1 Запуск приложения

Приложение запускается стандартным образом для Android-приложений через иконку на главном экране устройства. При первом запуске автоматически загружается начальная сцена (StartScene).

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
						47
Изм.	Лист	№ докум.	Подпись	Дата		

2.4.3.2 Основные методы и процедуры

Менеджеры

UIManager — основной менеджер интерфейса:

- ExitGame() — выход из игры;
- CloseMenu() — закрытие текущего меню;
- AudioControl() — переключение состояния музыки.

InteractionManager — менеджер взаимодействия с объектами:

— SetCurrentInteractable (InteractionObject obj) - установка текущего интерактивного объекта

- InteractWithCurrent() — взаимодействие с текущим объектом

Inventory — менеджер инвентаря:

- PutInEmptySlot(Item item) — добавление предмета в инвентарь;
- GetInventoryData() — получение списка предметов;
- GetTotalBonus() — расчет суммарного бонуса от всех предметов;
- ApplyInventoryData(string[] data) — применение данных с сохранения.

Система сохранения

Система сохранения реализована через класс SaveManager:

- SaveGame() — сохранение текущего состояния игры в JSON-файл
- LoadGame() — загрузка сохраненного состояния из JSON-файла

Вызов сохранения: SaveManager.Instance.SaveGame()

Вызов загрузки данных: SaveManager.Instance.LoadGame()

2.4.4 ВХОДНЫЕ И ВЫХОДНЫЕ ДАННЫЕ

2.4.4.1 Формат сохранения данных

Все игровые данные сохраняются в формате JSON в файл savegame.json.

Основные сохраняемые данные:

```
{  
  "moonshards": 436,  
  "currentAspect": "Maid",  
  "currentMoonPhase": "WaxingCrescent",  
  "phaseTimer": 598.159912109375,  
}
```



```

"storeSlots": [ true, false, true, false, false, false, false, false, false, false, false, false],
"itemIDs": ["item_01", "item_04", "", "", "", "", "", "", "", "", "", "", ""],
"shardBonus": 10
}

```

Moonshards — количество ЛО у игрока.

currentAspect — аспект персонажа.

currentMoonPhase — фаза Луны.

phaseTimer — время фазы.

storeSlots — состояние каждой ячейки инвентаря (true – куплено, false – не куплено)

itemIDs — предметы в инвентаре и их ячейка.

shardBonus — общий бонус имеющихся предметов.

2.4.4.2 Входные данные

Данные от пользователя

- касания экрана (Touch);
- нажатия кнопок (Button clicks);
- перемещение персонажа (Joystick input).

Системные данные

- текущее системное время (для определения фазы Луны);
- уровень заряда батареи (для энергосберегающего режима);
- язык системы (для локализации).

2.4.4.3 Выходные данные

Графический вывод

- основной игровой экран;
- интерфейс пользователя (меню, инвентарь, статистика);
- анимации и визуальные эффекты.

Звуковой вывод

- фоновая музыка;
- звуковые эффекты взаимодействия.

Сохраненные данные

— файл savegame.json с текущим состоянием игры.

2.5 Руководство пользователя

2.5.1 ВВЕДЕНИЕ

2.5.1.1 Назначение руководства

Настоящее руководство пользователя предназначено для ознакомления игроков с игровым приложением «Лунные осколки» и оказания помощи в освоении всех функций и механик игры. [12][ЯЯ23]

2.5.1.2 Общая информация об игре

«Лунные осколки» — это казуальная мобильная игра в жанре головоломок, разработанная для устройств на базе операционной системы Android 8.0 (Oreo) и выше. Игра сочетает элементы коллекционирования, мини-игр и исследования мира, связанного с фазами Луны.

Основная цель игры — собрать Лунные осколки (ЛО), проходить мини-игры, изучать различные аспекты персонажа и исследовать локации в мире игры.

2.5.2 ОБЩЕЕ ОПИСАНИЕ ИГРЫ

2.5.2.1 Основные локации

Игра состоит из трех основных локаций:

1. Деревня Ильмала — главная локация с домами жителей и торговыми шатрами.
2. Святилище Неба — главный храм, где почитают Менес и Кернунноса.
3. Храм первого света — заброшенный храм Лунной Богини.

2.5.2.2 Основные элементы интерфейса

Основной интерфейс игры (рис. 23) включает следующие элементы:

- индикатор фазы Луны — расположенный в левом верхнем углу экрана;
- кнопка паузы — расположенная в правом верхнем углу экрана;

— джойстик управления — расположенный в левом нижнем углу экрана.



Рисунок 23 — Интерфейс игры

2.5.3 НАЧАЛО РАБОТЫ С ИГРОЙ

2.5.3.1 Запуск игры

Для запуска игры:

1. Найдите иконку игры «Лунные осколки» (рис. 24) на главном экране вашего смартфона.



Рисунок 24 — Иконка игры

2. Нажмите на иконку для запуска приложения.

2.5.3.2 Начальная сцена

После запуска игры вы увидите начальную сцену (рис. 25) со следующими элементами:

- название игры;
- кнопка «Начать игру»;
- кнопка «Обучение»;
- кнопка «Выход из игры».

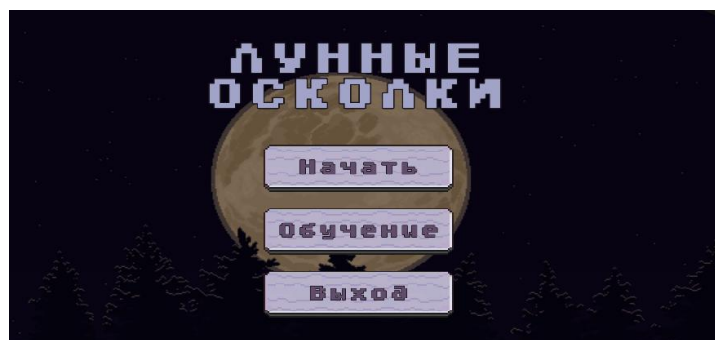


Рисунок 25 — Начальная сцена

2.5.3.3 Настройки аудио

В меню паузы (рис. 26) вы можете настроить аудио:

1. Нажмите на кнопку в верхнем правом углу.
2. В меню найдите кнопку «Музыка».
3. Используйте переключатель для включения/отключения фоновой музыки.



Рисунок 26 — Меню паузы, пример переключения музыки

2.5.4 ОСНОВНЫЕ ФУНКЦИИ ИГРЫ

2.5.4.1 Перемещение персонажа

Для перемещения персонажа:

1. Используйте джойстик управления в левом нижнем углу экрана.
2. Перемещайте персонажа между локациями: Деревня Ильмала, Святилище Неба, Храм первого света.

Пример джойстика на рисунке 23, нижний левый угол.

2.5.4.2 Взаимодействие с объектами

Для взаимодействия с объектами:

1. Подойдите к объекту (алтарь, статуя, лавка).
2. Нажмите на появившуюся кнопку взаимодействия в правой стороне экрана (рис. 27).

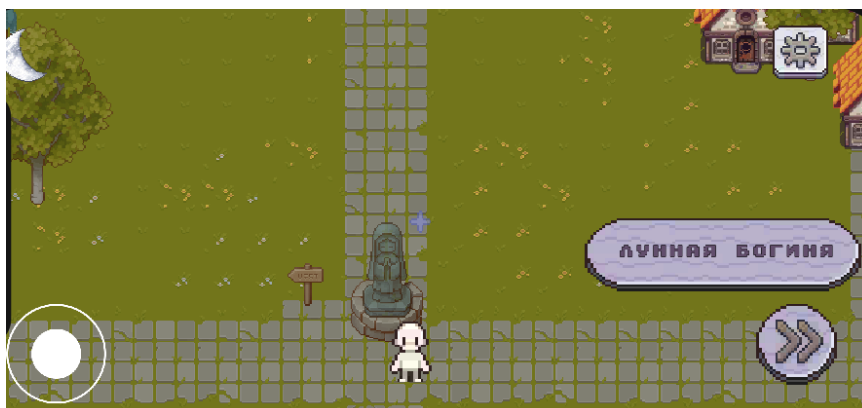


Рисунок 27 — Кнопка взаимодействия

3. Следуйте инструкциям на экране.

2.5.4.3 Фазы Луны

Игра содержит систему фаз Луны, которая влияет на игровой процесс:

- фазы Луны меняются автоматически со временем;
- каждая фаза имеет уникальные особенности и влияет на игру;
- текущая фаза отображается в индикаторе в левом верхнем углу.

Пример индикатора фазы на рисунке 23, верхний левый угол.

2.5.4.4 Мини-игры

Игра содержит две мини-игры:

- «Зеркальца» — головоломка, где необходимо направить луч света через систему зеркал;

Пример вида головоломки «Зеркальца» на рисунке 28.



Рисунок 28 — Уровень «Зеркальца»

— «Камушки-Кумушки» — игра «три в ряд».

Пример вида мини-игры «Камушки-Кумушки» на рисунке 29.



Рисунок 29 — Уровень «Камушки-Кумушки»

2.5.4.5 Лунные осколки

Лунные осколки (ЛО) — основная валюта игры:

- собирайте ЛО за выполнение мини-игр;
- используйте ЛО для покупки предметов в лавке;
- количество ЛО отображается в интерфейсе игры.

2.5.4.6 Предметы и инвентарь

Игра содержит систему предметов и инвентаря (рис. Е.8):

- собирайте предметы в ходе игры
- просматривайте предметы в инвентаре
- используйте предметы для получения бонусов

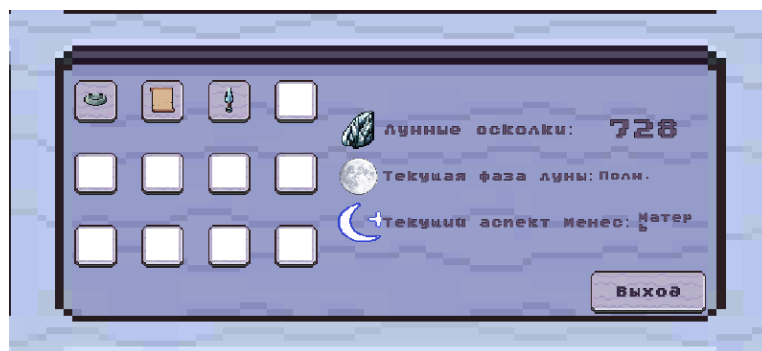


Рисунок 30 — Инвентарь

2.5.5 ОБУЧЕНИЕ

2.5.5.1 Доступ к обучению

Обучение доступно с начальной сцены игры:

1. Нажмите на кнопку «Обучение» на начальном экране (рис. 25)
2. Выберите нужную тему из меню обучения (рис. 31)

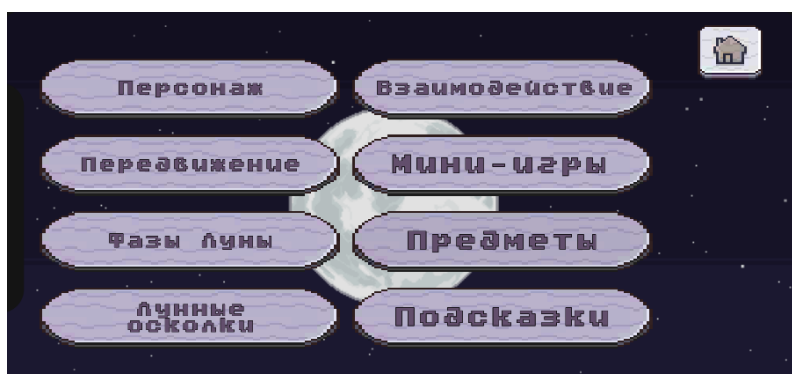


Рисунок 31 — Меню выбора темы обучения

2.5.5.2 Темы обучения

Обучение состоит из следующих тем:

1. Персонаж — описание возможностей персонажа и его аспектов.
2. Взаимодействие — как взаимодействовать с объектами в игре.
3. Перемещение — управление персонажем и переходы между локациями.
4. Мини-игры — правила и особенности мини-игр «Зеркальца» и «Камушки-Кумушки».
5. Фазы луны — информация о системе фаз Луны и их влиянии на игру.
6. Предметы — описание системы предметов и инвентаря.

7. Лунные осколки — как собирать и использовать Лунные осколки.
8. Подсказки — как использовать систему подсказок в игре.

2.5.5.3 Прохождение обучения

Для прохождения обучения:

1. Выберите тему из меню обучения.
2. Ознакомьтесь с информацией.
3. Нажмите «Далее» для перехода к следующему шагу (рис. 32).
4. Нажмите «Заккрыть» для выхода из обучения по теме.



Рисунок 32 — Меню выбранной темы

2.5.6 РЕШЕНИЕ ТИПОВЫХ ПРОБЛЕМ

2.5.6.1 Проблемы с сохранением игры

Если игра не сохраняется:

- убедитесь, что у вас достаточно места на устройстве (требуется 200–300 мБ свободного места);
- проверьте, что игра имеет права на запись данных;
- перезапустите игру и попробуйте сохранить прогресс снова.

2.5.6.2 Проблемы с аудио

Если отсутствует звук:

- проверьте, не выключен ли звук в системных настройках;
- убедитесь, что фоновая музыка включена в меню игры;
- проверьте, не поврежден ли файл аудио (переустановите игру при необходимости).

2.5.6.3 Проблемы с производительностью

Если игра тормозит:

- убедитесь, что ваше устройство соответствует минимальным требованиям;
- закройте другие приложения, которые могут потреблять ресурсы;
- попробуйте перезапустить игру.

2.6 Экономическое обоснование проекта

Расчет сметы затрат на разработку программы

Для расчета сметы затрат составлен проект выполнения работ по созданию программы. Он представляет собой перечень мероприятий, которые необходимо выполнить, чтобы разработать и внедрить мобильную игру «Лунные осколки».

Составление проекта выполнения работ

Работы перечислены в требуемой последовательности с установленной продолжительностью каждого этапа, данные в таблице 7.

Таблица 7 — Проект выполнения работ по созданию программы

Наименование этапов	Продолжительность, дни
Получение задания на разработку программы	1
Анализ предметной области	5
Проектирование концептуальной модели	4
Проектирование графического интерфейса программного приложения	3
Разработка функциональных возможностей программы	15
Тестирование программы	1
Эксплуатация, сдача проекта	1

Общие затраты времени на разработку программы определены как сумма продолжительности работ и составляют 30 дней.

Расчет материальных затрат

В составе материальных затрат по разработке программы отражена стоимость:

- приобретаемых материалов, которые являются необходимым компонентом при проведении работ;
- покупной энергии, расходуемой на производственные и хозяйственные нужды (в часах).

Рассчитываем затраты на эксплуатационные материалы, исходные данные представлены в таблице 8, в ней же и результаты расчетов.

Таблица 8 — Затраты на эксплуатационные материалы

Наименование материала	Количество	Цена, руб.	Сумма, руб.
Накопитель (шт.)	1	500	500
Интернет (дней)	40	28	1123,8
Бумага (уп.)	2	209	418
Ручка (шт.)	1	30	30
Итого (С м)			2071,8

Интернет (дней) рассчитывается согласно тарифному плану:

Тариф с абонентской платой:

$$\text{Цена} = \text{Абонентская плата} / \text{Количество дней в месяце} \quad (2)$$

$$\text{Сумма, руб.} = \text{Цена} * \text{Общие затраты времени на разработку} \quad (3)$$

Тариф по использованному трафику:

В столбец Цена в таблице 2, указывается цена за 1 день по тарифному плану.

$$\text{Сумма, руб.} = \text{Цена 1 дня} * \text{Количество дней разработки} \quad (4)$$

Расчёт стоимости электроэнергии:

Персональный компьютер будет использоваться 30 дней по 8 часов в день, то есть 240 часов.

Исходные данные:

- потребляемая мощность — 0,30 кВт/ч;
- время работы на ПК — 240 ч;
- тариф по электроэнергии — 4,9 руб. /кВт или 3,9 руб. /кВт (для домов в городских населенных пунктах, оборудованных стационарными электрическими плитами или отопительными установками; для сельских населенных пунктов).

Рассчитываем стоимость электроэнергии (Сэл.) по формуле 4:

$$C_{\text{эл.}} = P * t_{\text{раб.}} * Ц, \quad (5)$$

где P — потребляемая мощность, кВт/ч;

t раб. — время работы на ПК, час;

Ц — цена за 1 кВт/ч, руб

$$C_{\text{эл.}} = 0,30 * 240 * 4,9 = 252,8 \text{ руб.}$$

Рассчитываем сумму материальных затрат (С м.з.) по формуле 5:

$$C_{\text{м.з.}} = C_{\text{м.}} + C_{\text{эл.}}, \quad (6)$$

$$C_{\text{м.з.}} = 2253 + 252,8 = 2605,8 \text{ руб.}$$

Расчет затрат на оплату труда

Заработная плата — это материальное вознаграждение за труд в зависимости от степени квалификации работника, уровня сложности, количества и качества выполняемой работы, а также условий, в которых выполняется работа.

Количество труда — характеристика совокупности затрат мускульной и нервно-эмоциональной энергии работника в процессе его трудовой деятельности. Оно измеряется количеством произведенной продукции или отработанного времени.

Качество труда — степень сложности, напряженности и народнохозяйственного значения труда. На качество труда влияют и условия труда: привлекательность, тяжесть, наличие вредных для здоровья технологических процессов. Одна из важнейших категорий, которая влияет на объемы выпуска продукции. Характеризуется соотношением объема произведенной продукции и затрат трудовых ресурсов.

Для оплаты труда программиста чаще всего используется повременная зарплата.

Повременная форма оплаты труда — это оплата труда за отработанное время.

Для определения заработной платы существует тарифная система.

Тарифная система — совокупность норм и нормативов, обеспечивающих дифференциацию оплаты труда, исходя из различий в сложности выполняемых работ, условий труда, интенсивности и характера труда.

Тарифная система включает следующие элементы:

— тарифная сетка — это совокупность тарифных разрядов работ (профессий, должностей), определенных в зависимости от сложности работ и требований к квалификации работников с помощью тарифных коэффициентов;

— тарифная ставка — это закрепленная документально величина вознаграждения за достижение трудовой нормы той или иной степени трудности работником определенной квалификации за принятую единицу времени;

— единый тарифно-квалификационный справочник, который предназначен для тарификации работ, присвоения квалификационных разрядов рабочим, а также для составления программ по подготовке и повышению квалификации рабочих во всех отраслях и сферах деятельности;

— районный коэффициент — показатель, используемый при расчете заработной платы работника за труд в сложных климатических условиях.

Затраты на оплату труда включают заработную плату программиста.

Расчет заработной платы программиста в данной работе будем вести из расчёта минимальной оплаты труда, действующей в регионе.

Исходные данные:

- время работы над программой, 30 дн.;
- минимальная оплата труда (МРОТ), 27093 руб.;
- количество рабочих дней, 30 дн.;
- районный коэффициент, 1,15.

Рассчитываем заработную плату по тарифу по формуле 7:

$$З \text{ пл.} = \frac{\text{МРОТ}}{Т \text{ м.}} * Т * 1,15 \quad (7)$$

где З пл. - заработная плата, руб;

Т — время работы над программой, дн.;

МРОТ – минимальная оплата труда в регионе, руб.;

Т м. — количество дней за месяц, дн.

$$З \text{ пл.} = \frac{27093}{20} * 30 * 1,15 = 46735,425 \text{ руб.}$$

Расчет амортизационных отчислений

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
Изм.	Лист	№ докум.	Подпись	Дата		60

Для разработки мобильной игры «Лунные осколки» использовалась вычислительная техника в виде персонального компьютера, в который входит монитор и системный блок. Вычислительная техника входит в состав основных производственных фондов.

Производственные фонды — совокупность средств труда и предметов труда, необходимых для материального производства.

Основные производственные фонды — это средства труда, которые участвуют во многих производственных циклах, переносят свою стоимость частями по мере износа на готовую продукцию, сохраняя при этом натурально-вещественную форму.

Амортизация — это процесс переноса стоимости основных средств на стоимость произведенной и проданной конечной продукции по мере их износа, как материального, так и морального.

Амортизационные отчисления — это постепенное погашение стоимости основных фондов предприятия (зданий, оборудования, подвижного состава и т. п.), изнашивающихся в процессе работы и от времени.

Норма амортизации — это установленный размер амортизационных отчислений за определенный период времени, выраженный в процентах от балансовой стоимости основных фондов. Нормы амортизации устанавливаются для каждого вида основных средств и зависят от условий эксплуатации. Для компьютерного оборудования норма амортизации составляет 17%.

Рассчитываем амортизационные отчисления за время работы над проектом, исходные данные и результаты расчётов в таблице 9.

Таблица 9 — Амортизационные отчисления за время работы над проектом

Наименование основных производственных фондов	Стоимость ОПФ, руб.	Норма амортизации, %	Сумма амортизации, руб.
1. Системный блок	70000	17	11900
2. Монитор	25000	17	4250
Итого (А)			16150

А — сумма амортизационных отчислений

Сумма амортизационных отчислений (А) рассчитывается по формуле 8:

$$A = \frac{C_{п*H_{а}}}{K_{дг}} * K_{дп}, \quad (8)$$

где С п. — стоимость первоначальная, руб.;

Н а. — норма амортизации, в долях (% / 100);

К дг. — количество дней в году, 365 дней;

К дп. — количество дней работы над проектом.

Рассчитываем сумму амортизационных отчислений системного блока, А с.б.:

$$A_{с.б.} = \frac{70000 * 0,17}{365} * 40 = 1304,2 \text{ руб.}$$

Рассчитываем сумму амортизационных отчислений монитора, А м.:

$$A_{м.} = \frac{25000 * 0,17}{365} * 40 = 465,7 \text{ руб.}$$

$$A = A_{с.б.} + A_{м.} = 1304,2 + 465,7 = 1769,9 \text{ руб.}$$

Расчет стоимости разрабатываемой программы

Себестоимость — это сумма затрат данного предприятия на производство и реализацию продукции.

Себестоимость продукции формируется из следующих элементов:

- материальные затраты;
- затраты на оплату труда;
- амортизация основных фондов.

Рассчитываем себестоимость разрабатываемой программы по формуле 9:

$$C = C_{м.} + 3 \text{ пл.} + A, \quad (9)$$

где С — себестоимость, руб.;

С м. — материальные затраты, руб.;

3 пл. — затраты на оплату труда, руб.;

А — амортизационные отчисления, руб.

Рассчитываем себестоимость:

Исходные данные:

С м. — материальные затраты, 2605,8 руб.;

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
						62
Изм.	Лист	№ докум.	Подпись	Дата		

З пл. — затраты на оплату труда, 46735,425 руб.;

А — амортизационные отчисления, 1769,9 руб.;

$$C = 2605,8 + 46735,425 + 1769,9 = 51111,125 \text{ руб.}$$

Определяем структуру себестоимости разрабатываемой программы.

Структура себестоимости – процентное содержание элементов затрат к общей сумме себестоимости.

Таблица 10 — Структура себестоимости

Наименование статей затрат	Сумма, руб.	Структура, %
Материальные затраты	2605,8	6,07
Затраты на оплату труда	46735,425	90,51
Амортизационные отчисления	1769,9	3,43
Итого:	51111,125	100

Вывод: самую большую долю затрат на разработку программы составляет оплата труда, а самую малую долю — амортизационные отчисления. Несомненно, что интеллектуальный труд является более ёмким и более сложным видом умственной деятельности.

Расчет экономической эффективности внедрения программы

Экономическая эффективность — это соотношение результатов с затратами. Если разрабатываемая программа пользуется спросом, то можно ее продать.

В этом случае эффективность определяется возможной прибылью от реализации разработанной программы.

Для этого необходимо установить цену на разработанную программу.

Цена — денежное выражение стоимости товара. Возможная цена формируется в следующих рамках:

- себестоимость продукции;
- цены конкурентов;
- уникальные достоинства продукции.

При установлении цены используют полную сбытовую себестоимость товарной продукции, она включает производственную себестоимость и внепроизводственные расходы.

Внепроизводственные (коммерческие) расходы включают затраты, связанные с реализацией продукции (расфасовка, упаковка, отгрузка, реклама, маркетинговые исследования, комиссионные; хранение, транспортировка), а также различного рода отчисления и платежи.

Полная себестоимость рассчитывается по формуле 10:

$$C_{п.} = C + P_{в.}, \quad (10)$$

где $C_{п.}$ — полная себестоимость единицы продукции, руб.;

C — себестоимость единицы продукции производственная, руб.;

$P_{в.}$ — расходы внепроизводственные (7% от производственной себестоимости), руб.

Исходные данные:

— производственная стоимость — 51111,125 руб.,

— расходы внепроизводственные — 3577,8 руб. (7 % от производственной себестоимости);

$$C_{п.} = 51111,125 + 3577,8 = 54688,9 \text{ руб.}$$

Вывод: таким образом, полная себестоимость готового программного продукта составила 54688,9 рублей.

2.7 Техника безопасности и охрана труда

Охрана труда — это система мер, направленных на сохранение жизни и здоровья работников в процессе работы. Включает в себя правовые, социально-экономические, организационные, технические, санитарно-гигиенические, лечебно-профилактические, реабилитационные и иные мероприятия.

Техника безопасности при разработке программного обеспечения

1. Допуск к работе

— к работе допускаются лица не моложе 18 лет, прошедшие медицинский осмотр, вводный инструктаж и проверку знаний по охране труда и электробезопасности.

2. Общие обязанности

— соблюдать правила внутреннего трудового распорядка и настоящую инструкцию;

— знать и уметь оказывать первую помощь при поражении электрическим током и других несчастных случаях;

— соблюдать пожарную безопасность: знать расположение огнетушителей и эвакуационных выходов;

— поддерживать чистоту и порядок на рабочем месте.

3. Перед началом работы

— проверить исправность оборудования, наличие защитного заземления, правильность подключения к сети;

— осмотреть и отрегулировать рабочее место по эргономическим нормам: высота стола и кресла, расстояние до экрана (60–80 см), угол наклона монитора, отсутствие бликов;

— протереть экран специальной салфеткой;

— включать оборудование в строгой последовательности: блок питания → периферия → системный блок;

— не приступать к работе при неисправностях, отсутствии заземления или первичных средств пожаротушения.

4. Во время работы

— соблюдать режим труда и отдыха: через каждые 45–60 мин — перерыв 5–10 мин; выполнять гимнастику для глаз, шеи, рук;

— держать вентиляционные отверстия устройств открытыми;

— не касаться задней панели системного блока при включенном питании;

— не переключать интерфейсные кабели при включенном питании;

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
						65
Изм.	Лист	№ докум.	Подпись	Дата		

- не допускать попадания влаги на оборудование;
- не загромождать поверхности устройств бумагами и посторонними предметами;

- при длительном перерыве (более 10 мин) корректно закрыть задачи; отключать питание — только если находишься вблизи (<2 м) от устройства.

5. В аварийных ситуациях

- при травме или ухудшении самочувствия — немедленно прекратить работу, сообщить руководителю и оказать первую помощь;

- при задымлении/возгорании — выключить питание, эвакуировать людей, сообщить в службу пожарной охраны (101), при отсутствии угрозы жизни — использовать огнетушитель;

- при неисправности оборудования — немедленно прекратить работу и сообщить ответственному лицу.

6. По окончании работы

- закрыть все активные задачи, выключить оборудование;
- убрать документацию, привести рабочее место в порядок;
- проветрить помещение, вымыть руки, выключить свет.
- удостовериться в противопожарной безопасности и закрыть помещение.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта на тему «Разработка игры на смартфон» была поставлена цель разработать мобильную игру. Проект включал в себя несколько ключевых задач, таких как:

- выполнить анализ предметной области разработки мобильных игр;
- изучить теоретические аспекты проектирования и создания игровых приложений для смартфонов;
- проанализировать средства разработки и выбрать оптимальный игровой движок, среду программирования и графический редактор;
- выполнить планирование и проектировку приложения, разработать концепт-документ и техническое задание;
- реализовать программный код, визуальное оформление и функциональные механики игрового приложения в выбранной среде разработки;
- провести функциональное тестирование готового продукта;
- разработать руководство программиста;
- составить руководство пользователя;
- выполнить экономическое обоснование проекта, рассчитав себестоимость разработки и эффективность внедрения;
- рассмотреть вопросы техники безопасности и охраны труда при разработке программного обеспечения.

На предпроектной стадии разработки был проведен анализ предметной области и популярных игр-головоломок. В результате анализа было решено создать мобильную игру в жанре казуальных головоломок, отличительной чертой которой станет уникальный сеттинг, созданный на основе балтийских и славянских мифологий, с различными заданиями и уровнями.

Изучение теоретических материалов по области мобильной разработки и мобильных игр позволило выделить наиболее важные и актуальные аспекты, на которые необходимо опираться для создания успешного проекта.

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
Изм.	Лист	№ докум.	Подпись	Дата		67

После анализа предметной области был произведен поиск наиболее подходящих для проекта средств разработки: платформа для разработки игр Unity 2023.1, редактор кода Visual Studio 2022, редактор изображений SAI, а также вспомогательные инструменты, такие как GitHub, Microsoft Word, Microsoft PowerPoint, для контроля версий, оформления документации и визуального представления соответственно.

На стадии проектировки приложение получило подробный концепт-документ, с описанием характеристик игры, механик и визуала, и техническое задание, полностью описывающее все детали, которые должна содержать готова игра.

В ходе реализации были разработаны визуальное оформление локаций, интерфейса, головоломок и персонажа с его анимациями. После было разработано множество скриптов, наполняющих игру интерактивностью. Готовый проект включает в себя все установленные техническим заданием механики и условия: мини-игру «Зеркальца» и «Камушки-Кумушки», механики смены фаз Луны, смены аспекта персонажа, коллекционные предметы, музыкальное сопровождение, сохранение прогресса игры, единый визуальный стиль.

Было проведено функциональное тестирование и тестирование производительности, по итогам которого можно сказать, что игровое приложение соответствует техническим требованиям, заявленными в техническом задании.

После завершения разработки проекта была подготовлена сопроводительная документация, включая в себя разработаны руководство программиста и руководство пользователя.

Руководство программиста подробно описывает структуру проекта, архитектуру программного кода и конфигурации настроек. Документ предназначено для разработчиков, участвующих в разработке игры.

Руководство пользователя описывает пользовательские сценарии, игровые механики, интерфейс и алгоритмы взаимодействия с приложением. Служить инструкцией для эксплуатации игры и предназначено для пользователей и игроков «Лунных осколков».

В рамках экономической части дипломного проекта был произведен расчет сметы затрат на разработку и внедрение разработанного мобильного приложения. Определены материальные затраты, затраты на оплату труда программиста и амортизационные отчисления за использование основных производственных фондов. Итоговая производственная себестоимость разработанного программного продукта составила 54688,9 руб.

В разделе «Техника безопасности и охрана труда» были описаны организационные и санитарные требования к рабочему месту программиста, правила допуска и обязанности работника к организации рабочего процесса, соблюдаемые на протяжении всей работы.

По результатам проведенной разработки поставленная цель: разработка игрового приложения на смартфон, и сопутствующие задачи были успешно выполнены. Конечным результатом стал игровой программный продукт «Лунные осколки», соответствующий установленным требованиям.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 2.105–95. Единая система конструкторской документации. Общие требования к текстовым документам [Текст]. — Издание официальное. — М.: Стандартиформ, 2011. — Апрель.

2. ГОСТ 7.32–2001. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — М.: ИПК Издательство стандартов, 2001.

3. ГОСТ 19.106–78. Единая система программной документации. Требования к программным документам, выполненным печатным способом [Текст]. — Издание официальное. — М.: Стандартиформ, 2010. — Январь.

4. ГОСТ 19.504–79. Единая система программной документации. Руководство программиста. Требования к содержанию и оформлению [Текст]. — М.: Издательство стандартов, 1979.

5. ГОСТ 19.701–90. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения [Текст]. — М.: Издательство стандартов, 1990.

6. ГОСТ 24.301–80. Система технической документации на АСУ. Общие требования к выполнению текстовых документов [Текст]. — М.: Издательство стандартов, 1980.

7. ГОСТ 2.601–2006. Единая система конструкторской документации. Эксплуатационные документы [Текст]. — М.: Стандартиформ, 2006.

8. ГОСТ Р 59795–2021. Система стандартов по информации, библиотечному и издательскому делу. Руководство пользователя. Требования к содержанию и оформлению [Текст]. — М.: Стандартиформ, 2021.

9. ОСТ 29.115–88. Оригиналы авторские и текстовые издательские. Общие технические требования [Текст]. — 1988.

10. Архипова, Н. А. Геймдизайн и проектирование компьютерных игр [Текст] : учебное пособие / Н. А. Архипова, И. Е. Рогов. — Москва: РТУ МИРЭА,

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист
Изм.	Лист	№ докум.	Подпись	Дата		70

2025. — 103 с. — ISBN 978-5-7339-2442-7. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/book/493364> (дата обращения: 30.05.2026). — Режим доступа: для авториз. пользователей.

11. Гагарина, Л. Г. Технология разработки программного обеспечения [Текст] : учебное пособие / Л. Г. Гагарина, Е. В. Кокорева, Б. Д. Виснадул ; под ред. Л. Г. Гагариной. — М. : ИД «ФОРУМ» : ИНФРА-М, 2008. — 400 с.: ил. — (Высшее образование).

12. Диязитдинова, А. Р. Основы проектирования информационных систем [Текст] : учебное пособие / А. Р. Диязитдинова. — Самара: ПГУТИ, 2025. — 123 с. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/book/517382> (дата обращения: 03.06.2026). — Режим доступа: для авториз. пользователей.

13. Зубек, Роберт. Элементы гейм-дизайна. Как создавать игры, от которых невозможно оторваться [Текст] / Роберт Зубек; [пер. с англ. О. И. Перфильева]. — Москва : Эксмо, 2022. — 272 с.: ил. — (Мировой компьютерный бестселлер. Гейм-дизайн).

14. Колобов, Н. А. Активы в онлайн-играх как объект оценки [Текст] / Н. А. Колобов // Пермский финансовый журнал. — 2020. — № 1. — С. 48-62. — ISSN 2410-2776. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/journal/issue/347867> (дата обращения: 30.05.2026). — Режим доступа: для авториз. пользователей.

15. Куликова, Н. Ю. Онлайн-курс «Разработка компьютерных игр для мобильных устройств» при обучении школьников алгоритмизации и программированию [Текст] / Н. Ю. Куликова, Е. В. Данильчук, А. И. Малова // Известия Волгоградского государственного педагогического университета. — 2021. — № 6 (159). — С. 38-45. — ISSN 1815-9044. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/journal/issue/341093> (дата обращения: 24.03.2026). — Режим доступа: для авториз. пользователей.

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист 71
Изм.	Лист	№ докум.	Подпись	Дата		

16. Мочалова, Л. В. Методика оценки типографики и дизайна мобильных интерфейсов [Текст] / Л. В. Мочалова, И. Ю. Мамедова // Технологии и качество. — 2024. — № 2 (64). — С. 44–50. — ISSN 2587–6147. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/journal/issue/364712> (дата обращения: 30.05.2026). — Режим доступа: для авториз. пользователей.

17. Парамонов, А. И. Основы программной инженерии [Текст] : учебно-методическое пособие / А. И. Парамонов, Н. В. Лапицкая, С. Н. Нестеренков. — Минск: БГУИР, 2023. — ISBN 978-985-543-699-8. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/book/479501> (дата обращения: 04.06.2026). — Режим доступа: для авториз. пользователей.

18. Полуянов, В. П. Аспекты финансового потенциала игр для мобильных устройств [Текст] / В. П. Полуянов, М. А. Дунаевский // Донецкие чтения 2024: образование, наука, инновации, культура и вызовы современности : материалы IX Междунар. науч. конф. (Донецк, 15–17 окт. 2024 г.) : в 10 т. / под общ. ред. С. В. Беспаловой. — Донецк: ДонГУ, 2024. — Т. 2: Физические, химические, технические и компьютерные науки. Ч. 2. — С. 237. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/book/504699> (дата обращения: 30.05.2026). — Режим доступа: для авториз. пользователей.

19. Роджерс, Скотт. Level up! Руководство по созданию классных видеоигр [Текст] / Скотт Роджерс; [пер. с англ. А. Д. Голубевой]. — Москва : Эксмо, 2023. — 528 с.: ил. — (Мировой компьютерный бестселлер. Гейм-дизайн).

20. Чернов, Е. А. Тестирование и верификация ПО [Текст] : учебное пособие / Е. А. Чернов, М. А. Овчинникова, Д. Е. Новичков. — Москва: РТУ МИРЭА, 2024. — ISBN 978-5-7339-2255-3. — Текст: электронный // Лань: электронно-библиотечная система. — URL: <https://e.lanbook.com/book/432665> (дата обращения: 03.06.2026). — Режим доступа: для авториз. пользователей.

					ПЗ – ЮУГК – 09.02.07 – 2026 - ДП	Лист 72
Изм.	Лист	№ докум.	Подпись	Дата		

21. Godot Engine — System requirements [Электронный ресурс]. — Режим доступа: https://docs.godotengine.org/en/stable/about/system_requirements.html (дата обращения: 11.10.2024).

22. Newzoo. Global Games Market Report 2025 [Электронный ресурс, перевод]. — Режим доступа: <https://newzoo.com/resources/trend-reports/newzoo-global-games-market-report-2025> (дата обращения: 30.05.2026).

23. Unity — System Requirements [Электронный ресурс]. — Режим доступа: <https://unity3d.com/ru/unity/system-requirements> (дата обращения: 11.10.2024).

24. Unity. Разработка вашей первой игры с помощью Unity и C# [Электронный ресурс]: статья. — Режим доступа: <https://learn.microsoft.com/ru-ru/archive/msdn-magazine/2014/august/unity-developing-your-first-game-with-unity-and-csharp> (дата обращения: 10.11.2024).

25. Unity. Руководство: основы Unity [Электронный ресурс]: статья. — Режим доступа: <https://docs.unity3d.com/ru/530/Manual/UnityBasics.html> (дата обращения: 10.11.2024).

26. Unreal Engine. Hardware and Software Specifications [Электронный ресурс]. — Режим доступа: <https://dev.epicgames.com/documentation/en-us/unreal-engine/hardware-and-software-specifications-for-unreal-engine> (дата обращения: 11.10.2024).

27. Visual Studio: IDE и редактор кода для разработчиков и групп, работающих с программным обеспечением [Электронный ресурс]. — Режим доступа: <https://visualstudio.microsoft.com/ru/> (дата обращения: 11.10.2024).

28. Unity Profiler. User Manual [Электронный ресурс]. — Режим доступа: <https://docs.unity3d.com/Manual/Profiler.html> (дата обращения: 04.06.2026).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Таблица А.1 — Сравнение успешных мобильных игр

Параметр	Candy Crush Saga	Township	Clash Royale
Жанр	Головоломка	Симулятор, стратегия	Карточная стратегия
Стиль	Яркий, минималистичный	Мультяшный, красочный	Мультяшный, динамичный
Метод монетизации	Free-to-play + микротранзакции	Free-to-play + микротранзакции	Free-to-play + микротранзакции
Технические требования	Низкие (подходит для бюджетных устройств)	Низкие (оптимизирована под слабые устройства)	Низкие (оптимизирована под слабые устройства)
Длина сессий	Короткие (5–10 минут)	Длительные (15–30 минут)	Короткие (3–10 минут)
Целевая аудитория	Казуальные игроки, все возрасты	Любители симуляторов и стратегий	Любители стратегий и PvP (8+)
Движок	Unity	Unity	Unity

Таблица А.2 — Сравнение движков [ЯЯ24]

Критерий	Unity	Unreal Engine	Godot
Целевая аудитория	Подходит для всех уровней разработчиков	Опытные пользователи и студии	Независимые разработчики и энтузиасты
Язык программирования	C#	C++ (основной), Blueprints (визуальное программирование)	GScript (похож на Python), C#, Visual Scripting
Наличие ассетов	Огромный выбор готовых ассетов в Asset Store	Меньше ассетов, но есть качественные решения	Ограниченное количество, фокус на создание собственных ассетов
Инструменты оптимизации	Profiler, Addressables, Occlusion Culling, Texture Compression	Scalability Settings, Mobile Preview, Chaos Physics	Lightweight Architecture, Dynamic Fonts, Remote Debugging
Документация и комьюнити	Исчерпывающая документация, множество tutorиалов и активное сообщество	Хорошая документация, но сложнее для новичков	Меньше ресурсов, но активно развивающееся сообщество
Порог входа	Низкий благодаря интуитивному интерфейсу и C#	Высокий из-за сложности C++ и большого количества функций	Средний, но требует времени на освоение системы узлов
Оптимизация под мобильные устройства	Отличная оптимизация для мобильных игр (2D и 3D)	Подходит для высококачественных проектов, но может быть избыточным для простых игр	Легковесный движок, но требует ручной оптимизации для сложных проектов
Лицензия и стоимость	Бесплатный до \$100k дохода, затем 5% комиссия	Бесплатный до \$1 млн дохода, затем 5% комиссия	Полностью бесплатный (MIT License)

Таблица А.3 — Сравнение графических редакторов [ЯЯ25]

Критерий	Adobe Photoshop	PaintTool SAI
Целевая аудитория	Профессионалы, дизайнеры, фотографы	Новички, художники, иллюстраторы
Интерфейс	Комплексный, многофункциональный	Минималистичный, интуитивный
Инструменты рисования	Широкий набор кистей, текстур, фильтров	Базовый набор кистей с естественной реакцией на давление
Работа со слоями	Многослойная система с продвинутыми возможностями	Простая многослойная система с базовыми режимами смешивания
Производительность	Требует мощного оборудования для работы с большими файлами	Легковесный, работает на слабых компьютерах
Лицензия и стоимость	Платная подписка	Бесплатная лицензия для личного использования
Обучение	Высокий порог входа, требуется время на освоение	Низкий порог входа, интуитивное управление
Преимущества	Мощные инструменты, универсальность, поддержка множества форматов	Простота, легковесность, бесплатная лицензия, удобство для новичков
Недостатки	Высокая стоимость, сложность освоения, требовательность к оборудованию	Ограниченный функционал, отсутствие сложных эффектов и фильтров

Концепт-документ мобильной игры «Лунный осколки»

Название: «Лунные осколки»

Жанр: казуальная, головоломки

Сеттинг: мистические земли, деревни, святилища, заброшенные храмы, магия, ритуалы, таинственность места

Платформа: Android, iOS

Возрастное ограничение: 6+

Вдохновители

Candy Crush Saga

Бесплатная игра типа «три в ряд» с уровнями.

Механики:

- соединение трех элементов одного типа;
- цель уровня: очки или задачи;
- ограничения: ходы, жизни.

Дизайн

Цветовая палитра: Яркие, насыщенные цвета для объектов (ассоциации с фруктами) и светлые тона для фона.

Формы: Закругленные, мягкие очертания.

Стиль: Мультяшный минимализм.

Целевая аудитория

Описание целевой аудитории Candy Crush Saga приведено в таблице Б.1.

Таблица Б.1 — Портрет ЦА игры Candy Crush Saga

Характеристика	Описание
Пол	Преимущественно женщины (60–70%)
Возраст	Основной сегмент: 25–45 лет. Дополнительно: 18–24 года и 45+
География	США, Европа (основные рынки), Азия, Латинская Америка, Африка (растущие рынки).
Уровень дохода	Средний класс (основная аудитория), низкий и высокий уровень дохода (меньшие сегменты).
Поведение	Казуальные игроки, платящие игроки, социальные игроки.
Интересы	Любовь к простым, красочным играм, желание расслабиться, соревновательный дух.

Сильные и слабые стороны

Описание сильных и слабых сторон игры Candy Crush Saga приведено в таблице Б.2.

Таблица Б.2 — Сильные и слабые стороны Candy Crush Saga

Преимущества	Недостатки
Простота геймплея Яркий и привлекательный дизайн Монетизация через микротранзакции Социальные взаимодействия Удобство для мобильных устройств Разнообразие уровней Психологические триггеры	Ограниченная глубина геймплея Отсутствие уникальности Зависимость от монетизации Высокая сложность на поздних уровнях Агрессивная реклама

Brain Test: Хитрые головоломки

Мобильная игра в жанре головоломки от Unico Studio, основанная на нестандартном мышлении и абсурдных решениях, которые нарушают логику типичных puzzle-игр.

Механики

— интерактивность всех элементов: текст, фон, интерфейс — всё может быть частью решения.

— нарушение условностей: требуется взаимодействие с ОС устройства или интерфейсом игры.

— семантические ловушки: каламбуры, двойной смысл вопросов, визуальные подсказки.

— прогрессивная сложность: каждая головоломка может вводить уникальные правила без явных инструкций.

Дизайн

Цветовая палитра: Приглушенные, пастельные тона для фона, яркие акценты для активных объектов.

Формы: Закругленные, «мягкие» контуры без острых углов.

Стиль: Упрощенный карикатурный минимализм, напоминающий рисунки «от руки в тетради».

Целевая аудитория

Описание целевой аудитории игры Brain Test приведено в таблице Б.3.

Таблица Б.3 — Целевая аудитория игры Brain Test: Хитрые головоломки

Критерий	Описание
Пол	Преимущественно женщины и мужчины равномерно
Возраст	12–45 лет (основной сегмент), дети и пожилые люди (второстепенный)
География	Мировая аудитория, особенно популярна в странах с высоким уровнем мобильного использования (США, Европа, Азия).
Уровень дохода	Средний класс (основная аудитория), низкий уровень дохода (меньший сегмент)
Поведение	Казуальные игроки, любители загадок, семейные пары, родители с детьми

Сильные и слабые стороны

Описание сильных и слабых сторон игры Brain Test приведено в таблице Б.4.

Таблица Б.4 — Сильные и слабые стороны игры Brain Test

Сильные стороны	Слабые стороны
Простота освоения и интуитивный геймплей Уникальные и нестандартные задачи Юмор и неожиданные решения Большое количество задач Регулярные обновления контента	Зависимость от рекламы может раздражать игроков Однотипность механик на поздних уровнях Ограниченная реиграбельности Недостаток долгосрочной мотивации Высокая конкуренция среди похожих игр

Two Dots

Логическая игра, где игрок соединяет цветные точки для выполнения заданий.

Механики

- соединение точек одинакового цвета;
- цель уровня (очки, задания);
- ограничение уровня (ходы);
- временное ограничение попыток (жизни);
- дополнительные предметы на поле.

Дизайн

Цветовая палитра: Пастельные тона для фона, яркие и контрастные для интерактивных элементов.

Формы: Упрощенные геометрические очертания.

Стиль: Плоский дизайн с мультяшными элементами.

Целевая аудитория

Описание целевой аудитории игры Two Dots приведено в таблице Б.5.

Таблица Б.5 — Целевая аудитория Two Dots

Критерий	Описание
Пол	Преимущественно женщины (60–70%), также мужчины (30–40%).
Возраст	Основной сегмент: 18–45 лет. Второстепенный сегмент: подростки (12–17 лет)
География	США, Европа, страны Азии и Латинская Америка
Уровень дохода	Средний класс (основная аудитория), низкий уровень дохода (меньший сегмент)
Поведение	Казуальные игроки, которые ценят расслабляющий геймплей и короткие сессии
Интересы	Любители головоломок, простых механик и спокойного визуального стиля

Сильные и слабые стороны

Описание сильных и слабых сторон игры Two Dots приведено в таблице Б.6.

Таблица Б.6 — Сильные и слабые стороны Two Dots

Сильные стороны	Слабые стороны
Простой и интуитивный геймплей Минималистичный и стильный дизайн Яркая цветовая палитра привлекает внимание Регулярные обновления контента и уровней Подходит для коротких игровых сессий Эффективная модель free-to-play	Однообразие механик на поздних уровнях Ограниченная глубина стратегии Недостаток уникальных механик для выделения игры Конкуренция с похожими играми

С приведенных игр-вдохновителей можно выделить то, что людей привлекает минималистичный дизайн, простота механик и частые события. Отталкивает наличие навязчивой рекламы, однотипность и зависимость от монетизации. Целевая аудитория у данных игр: преимущественно женщины в возрасте от 12 до 40 лет со средним уровнем дохода.

Целевая аудитория

Данные для определения целевой аудитории взяты из бесплатных статей портала Newzoo [22], результаты представлены в таблице Б.7.

Таблица Б.7 — Данные целевой аудитории проекта «Лунные осколки»

Сегмент	Возраст	Пол	География	Уровень дохода
Основной	18–40 лет	Женщины (60–70%)	Россия и СНГ, Балтийские страны, Европа	средний
Второстепенный	12–17 лет	Мужчины (30–40%)	США и страны Азии	низкий

Механики

Цель игры заключается в коллекционировании предметов за счет прохождения головоломок и покупок. Пример игрового цикла приведен на рисунке Б.1.



Рисунок Б.1 — Игровой цикл «Лунных осколков»

8. Лунные осколки

Лунные осколки (далее «ЛО») — внутриигровая валюта, получаемая игроком в качестве награды за прохождение головоломок.

Осколки можно потратить в Лавке, купив коллекционные предметы или смену фазы.

9. Головоломки

За головоломку игроку дается некоторое количество Лунных осколков в качестве награды.

— головоломка «Зеркальца»

Игрок должен повернуть зеркала, чтобы лунный луч попал на алтарь. Каждое зеркало поворачивается только в определённом направлении. После попадания луча на алтарь уровень считается пройденным.

— мини-игра «Камушки-Кумушки»

Игрок должен передвигать камни так, чтобы собрать 3 одинаковых в один ряд (горизонтальный или вертикальный), с целью достигнуть определённых целей уровня.

10. Предметы

Покупаются за ЛО в магазине, хранятся в инвентаре. Дают усиления наград за головоломки. Содержат в своем описании лор игры.

11. Инвентарь

Хранит в себе купленные игроком предметы. Дает возможность посмотреть предметы, имеющимися у игрока, а также их характеристики и описание.

12. Магазин

Предоставляет и хранит список предметов, которые можно хранить, а также те предметы, которые были куплены. Можно просматривать купленные предметы, не купленные предметы и их характеристики.

13. Передвижение персонажа: по сцене и между сцен;

14. Смена фаз и состояний персонажа:

— Новолуние (Дева) — быстрая, скорость передвижения повышенная;

— Полнолуние (Мать) — полная сил, средняя скорость передвижения;

— Старая Луна (Старуха) — ослабленная, пониженная скорость передвижения.

Фазы сменяются самостоятельно каждый 15 минут реального времени. При смене фазы меняется задний фон и индикатор сверху экрана.

Дизайн

Стиль: свободный пиксель-арт

Палитра: палитры для сцен игры обладают спокойными, неяркими, но контрастными цветами.

Примеры палитр:

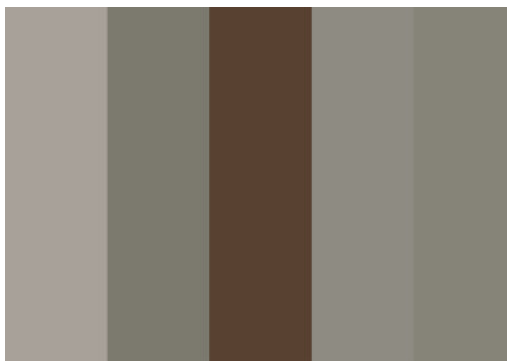


Рисунок Б.2 — Пример палитры 1

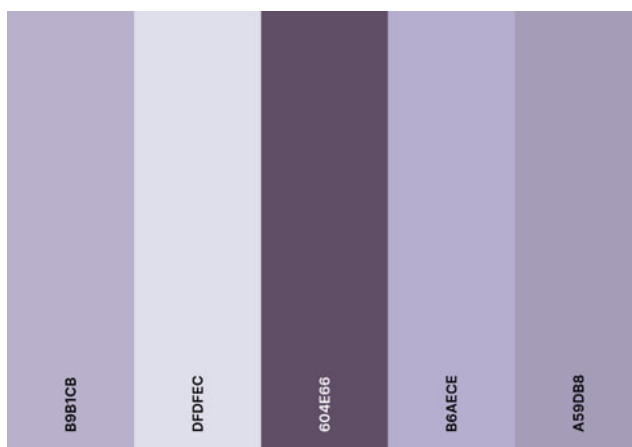


Рисунок Б.3 — Пример палитры 2

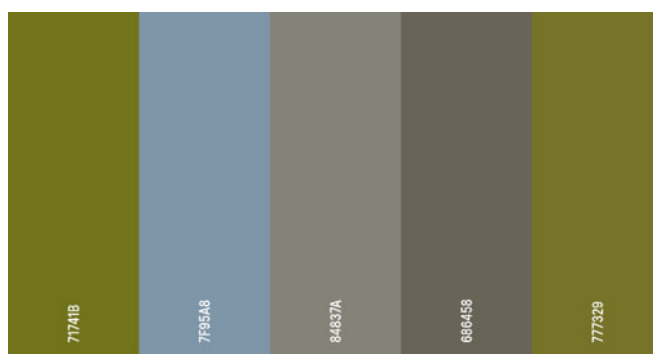


Рисунок Б.4 — Пример палитры 3

Локации

1. Деревня Ильмала — главная локация, небольшая деревня с домами жителей и торговыми шатрами.

Архитектура и окружение:

- дома из необожжённого кирпича и дерева;
- центральная площадь со статуей — Статуя Лунной Богини;
- слева проход к Храму первого света, сверху — к Святилищу Неба.

2. Святилище Неба — главный храм в округе, где почитают Менес и Кернунноса.

Архитектура и окружение:

- двухчастное здание: восточный двор — солнечная сторона и западный зал — лунная сторона;
- алтарь Священных камней — активирует мини-игру «Камушки-Кумушки».

3. Храм первого света — храм Менес, заброшенный с момента ее исчезновения, сохранивший историю и атмосферу.

Архитектура и окружение:

- зал, в конце которого — Зеркальный алтарь;
- колонны с трещинами, частью обрушены.

Персонажи

Главный герой: Менес — главная героиня, за которую игрок проходит игру. Потерявшая силы Богиня Луны. Собирает Лунные осколки для восстановления. Зависит от фазы луны:

- Новолуние (Дева) — светло-серый плащ;
- Полнолуние (Мать) — серый плащ;
- Старая луна (Старуха) — темно-серый плащ.

Синопсис

В начале игры Менес пробуждается в полуразрушенном Храме Первого Света, лишённая памяти и почти всей магии. Она чувствует лишь слабый отклик своей силы — разбросанные по миру Лунные осколки, и желание защитить своих людей.

Сеттинг

Действие игры разворачивается в Похьёлье — мистической земле, некогда процветавшей под покровительством Богини Луны Менес. Это страна, где всё — от шелеста листьев до дыхания ветра — пронизано древней, живой энергией, именуемой Белым Светом. Люди Похьёльи жили в гармонии с природой, почитая циклы Луны и Солнца, празднуя восемь саббатов Колеса Года и черпая силу из священных ритуалов.

Сюжет

Менес просыпается от очень длительного сна в заброшенном святилище Луны. Она плохо помнит, что произошло до ее пробуждения, но чувствует, что сил практически нет. Менес понимает, то ей необходимо вспомнить что произошло, исправить ситуацию и вернуть то, что она потеряла. Поэтому богиня отправляется в путь.

При выходе из храма она встречает таинственного старца, бормочешего про силу Луны, возвращение и восход. При разговоре старец укажет Менес на дорожку, которая ведет к деревне, а после исчезнет.

У входа в деревню стоит мальчик, который рассказывает про пропавшего жителя, про дела в деревне и говорит быть аккуратней, ведь в округе начали происходить странные вещи.

В деревне можно найти Жрицу Киллики, возлюбленную потерявшегося охотника. Как оказалось, он ушел из деревни ради того, чтобы возложить подношения у одного из алтарей и не вернулся (Его можно найти, если свернуть с тропинки у Храма первого света). Она так же рассказывает, что в округе деревни часто можно найти раненых животных, разрушенные алтари, следы разрушений в лесу, что люди стали бояться, больше сомневаться в Керунносе, в Верховных и в возвращении Богини Луны, но надеются на это сильнее, чем никогда раньше.

Для возврата сил необходимо собрать Лунные осколки (ЛО) и отнести их в главный алтарь в начальном святилище.

После сбора всех осколков дается часть Лунного серпа — рукоять, которую необходимо предъявить у входа в Храм Небес, чтобы пройти в него

и встретится с Верховными, которые отдадут вторую часть серпа – острый полумесяц с пикой в вершине. Две части соберутся в Лунца – оружие Менес, для защиты от врагов, и к Богине вернутся ее силы.

Лор

Весь мир пронизывает особая энергия – Белый свет, все состоит из нее: животные, природа, пейзажи. В мире есть люди, которые в разной степени могут управлять этой энергией, если она в достаточной мере есть у них. Кто-то рождается с большим количеством энергии, кто-то накапливает ее с течением жизни, кто-то не может управлять ей вовсе. Но так или иначе, она протекает во всем и во всех.

Но в мире на каждое действие есть противодействие. Черные тени – бессознательная энергия, постоянно поглощающая Свет, не знающая меры в разрушении. Во времена, когда Тени начали сходить с ума, сила была запечатана в Безлунном мире для контроля порядка и сохранения баланса. Но вот граница истончается, Тени прорываются в Похьеле, а значит, нужно что-то предпринимать.

После главного праздника года Ламмас, Менес пропадает, запечатывает себя в Храме первого свет, тем самым распределяя силу на своей земле, что помогло запечатать печать, и надеется, что Кернуннос сможет поддерживать мир, до ее восстановления.

В.1 Скрипты персонажа

Скрипт PlayerMenes

```

public class PlayerMenes : MonoBehaviour
{
    public static PlayerMenes instance;
    private Dictionary<AspectMenes, float> speedOfAspect = new()
    {
        {AspectMenes.Maid, 5f },
        {AspectMenes.Mother, 4f },
        {AspectMenes.Crone, 3.5f }
    };

    [Header("Текущий аспект")]
    private AspectMenes currentAspect = AspectMenes.Maid;

    private Rigidbody2D rbPlayer;
    private SpriteRenderer srPlayer;
    private Animator animPlayer;

    public Joystick controller;
    private void Awake()
    {
        if (instance == null)
        {
            instance = this;
        }
        else
        {
            Destroy(gameObject);
        }
    }
    private void Start()
    {
        rbPlayer = GetComponent<Rigidbody2D>();
        srPlayer = GetComponent<SpriteRenderer>();
        animPlayer = GetComponent<Animator>();
    }
    private void FixedUpdate()
    {
        MovePlayer();
    }
    public void SetAspect(AspectMenes newAspect)
    {
        currentAspect = newAspect;
    }

    public AspectMenes GetCurrentAspect()
    {
        return currentAspect;
    }
    private void MovePlayer()
    {
        float horizontalMove = controller.Horizontal;
        float verticalMove = controller.Vertical;

        float speed = speedOfAspect[currentAspect];
        bool isWalking = false;

        if (Mathf.Abs(horizontalMove) < 0.1f && Mathf.Abs(verticalMove) < 0.1f)
        {

```

```

        horizontalMove = 0f;
        verticalMove = 0f;
        rbPlayer.velocity = Vector2.zero;
    }
    else
    {
        isWalking = true;
        Vector2 movment = new Vector2(horizontalMove, verticalMove).normalized;
        rbPlayer.velocity = movment * speed;
    }
    UpdateAnimation(horizontalMove, verticalMove, isWalking);
}
private void UpdateAnimation(float horizontal, float vertical, bool isWalking)
{
    animPlayer.SetFloat("Horizontal", horizontal);
    animPlayer.SetFloat("Vertical", vertical);
    animPlayer.SetBool("IsWalking", isWalking);
    if (horizontal < -0.1f)
    {
        srPlayer.flipX = true;
    }
    else if (horizontal > 0.1f)
    {
        srPlayer.flipX = false;
    }
}
}
[Serializable]
public enum AspectMenes
{
    Maid,
    Mother,
    Crone
}

```

Скрипт MenesManager

```

public class MenesManager : MonoBehaviour
{
    public static MenesManager instance;
    public PlayerMenes playerMenes;
    private int moonShard;

    private void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else { Destroy(gameObject); }
    }
    private void OnEnable()
    {
        SceneManager.sceneLoaded += OnSceneLoaded;
    }
    private void OnDisable()
    {
        SceneManager.sceneLoaded -= OnSceneLoaded;
    }
    private void OnSceneLoaded(Scene scene, LoadSceneMode mode)
    {
        SceneType levelType = FindObjectOfType<SceneIntializer>().sceneType;
        if (levelType == SceneType.FreeZone)
        {

```

```

        if (playerMenes == null)
        {
            playerMenes =
GameObject.FindGameObjectWithTag("Player").GetComponent<PlayerMenes>();
        }
    }
    public void SetNewAspect(AspectMenes newAspect)
    {
        if (playerMenes != null)
        {
            playerMenes.SetAspect(newAspect);
        }
        else
        {
            Debug.Log("Неназначен управляемый персонаж");
        }
    }
    public void TakeMoonshard(int countSharde)
    {
        Debug.Log($"Начисленно {countSharde}");
        moonShard += countSharde;
    }
    public void GiveMoonshard(int countShard)
    {
        if (countShard > 0 && countShard <= moonShard)
        {
            moonShard -= countShard;
            return;
        }
    }
    public void SetMoonshard(int moonShards)
    {
        this.moonShard = moonShards;
    }
    public int GetMoonshard()
    {
        return moonShard;
    }
}

```

В.2 Скрипты-менеджеры

Скрипт LevelSettings

```

public class LevelSettings : MonoBehaviour
{
    public static LevelSettings instance;

    [System.Serializable]
    public class SceneSettings
    {
        public SceneType levelType;
        public bool spawnPlayer = true;
    }
    public SceneSettings[] sceneSettings;
    private void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
        }
    }
}

```



```

        Destroy(gameObject);
    }
}
public void SetTypeScene(SceneType sceneType)
{
    foreach (var scene in sceneSettings)
    {
        if (scene.levelType == sceneType)
        {
            ApplySceneSettings(scene);
            return;
        }
    }
}
private void ApplySceneSettings(SceneSettings sceneSettings)
{
    string currentScene = SceneManager.GetActiveScene().name;

    if (sceneSettings.spawnPlayer)
    {
        SpawnPlayer();
    }

    if (AudioManager.instance != null)
    {
        AudioManager.instance.PlayMusicForScene(currentScene);
    }
}
private void SpawnPlayer()
{
    string spawnPointTag = SceneTransitionManager.instance.GetSpawnPointTag();

    GameObject spawnPoint = GameObject.FindGameObjectWithTag(spawnPointTag);
    if (spawnPoint != null)
    {
        GameObject player = GameObject.FindGameObjectWithTag("Player");

        if (player != null)
        {
            player.transform.position = spawnPoint.transform.position;
        }
    }
    else
    {
        Debug.Log("Точка спавна не найдена");
    }
}
}
public enum SceneType
{
    FreeZone,
    Puzzle,
    Menu
}

```

Скрипт SceneTransitionManager

```

public class SceneTransitionManager : MonoBehaviour
{
    public static SceneTransitionManager instance;

    private string lastSceneName = "";
    private string spawnPointTag = "SpawnPointMain";

    private void Awake()

```

```

{
    if (instance == null)
    {
        instance = this;
        DontDestroyOnLoad(gameObject);
    }
    else
    {
        Destroy(gameObject);
    }
}

public void TransitionToScene(string sceneName)
{
    SaveManager.SaveGame();
    lastSceneName = SceneManager.GetActiveScene().name;
    spawnPointTag = sceneName switch
    {
        "Village" when lastSceneName == "HolyOFLune" => "SpawnPointHoly",
        "Village" => "SpawnPointTemple",

        "HolyOFLune" => "SpawnPointHoly",
        "TempleOfSky" => "SpawnPointTemple",

        _ => "SpawnPointMain"
    };
    Debug.Log(spawnPointTag);
    SceneManager.LoadScene(sceneName);
}

public string GetSpawnPointTag()
{
    return spawnPointTag;
}
}

```

Скрипт MoonPhaseManager

```

public class MoonPhaseManager : MonoBehaviour
{
    public static MoonPhaseManager instance;

    private MoonPhase currentPhase;
    private SceneType currentSceneType;
    private float phaseDuration = 900f;
    private float timer = 0;

    [Header("Компоненты")]
    public Image background;
    public MenesManager menesManager;
    public Image icon;

    [Header("Фоны")]
    public Sprite waxingCrescentBackground;
    public Sprite fullMoonBackground;
    public Sprite waningCrescentBackground;

    [Header("Иконки лун")]
    public Sprite waxingCrescentIcon;
    public Sprite fullMoonIcon;
    public Sprite waningCrescentIcon;

    private void Awake()
    {
        if (instance == null)
        {

```

```

        instance = this;
        DontDestroyOnLoad(gameObject);

        currentPhase = MoonPhase.WaxingCrescent;
        timer = 0;
    }
    else { Destroy(gameObject); }
}
private void OnEnable()
{
    SceneManager.sceneLoaded += OnSceneLoaded;
}
private void OnDisable()
{
    SceneManager.sceneLoaded -= OnSceneLoaded;
}

private void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    SceneIntializer levelType = FindObjectOfType<SceneIntializer>();
    if (levelType != null)
    {
        currentSceneType = levelType.sceneType;

        if (currentSceneType == SceneType.FreeZone)
        {
            SetIconAndBackground();
        }
    }
    private void SetIconAndBackground()
    {
        if (background == null)
        {
            GameObject backgroundObject =
GameObject.FindGameObjectWithTag("Background");
            if (backgroundObject != null)
            {
                background = backgroundObject.GetComponent<Image>();
            }
            else
            {
                Debug.LogError("Background не найден на сцене!");
            }
        }

        if (icon == null)
        {
            GameObject iconObject = GameObject.FindGameObjectWithTag("IconMoon");
            if (iconObject != null)
            {
                icon = iconObject.GetComponent<Image>();
            }
            else
            {
                Debug.LogError("Icon не найден на сцене!");
            }
        }
    }
}
private void Update()
{
    if (currentSceneType == SceneType.FreeZone)
    {
        timer += Time.deltaTime;
    }
}

```

```

        if (timer > phaseDuration)
        {
            ChangeMoonPhase();
            timer = 0;
        }
    }

    private void ChangeMoonPhase()
    {
        currentPhase = (MoonPhase)(((int)currentPhase + 1) %
System.Enum.GetValues(typeof(MoonPhase)).Length);
        UpdateMoonPhase();
    }

    public float GetTimer()
    {
        return timer;
    }

    public MoonPhase GetCurrentPhase()
    {
        return currentPhase;
    }

    public void SetPhase(MoonPhase phase, float timer)
    {
        currentPhase = phase;
        this.timer = timer;
        UpdateMoonPhase();
    }

    private void UpdateMoonPhase()
    {
        switch (currentPhase)
        {
            case MoonPhase.WaxingCrescent:
                background.sprite = waxingCrescentBackground;
                icon.sprite = waxingCrescentIcon;
                menesManager.SetNewAspect(AspectMenes.Maid);
                break;

            case MoonPhase.FullMoon:
                background.sprite = fullMoonBackground;
                icon.sprite = fullMoonIcon;
                menesManager.SetNewAspect(AspectMenes.Mother);
                break;

            case MoonPhase.WaningCrescent:
                background.sprite = waningCrescentBackground;
                icon.sprite = waningCrescentIcon;
                menesManager.SetNewAspect(AspectMenes.Crone);
                break;
        }
    }
}

public enum MoonPhase
{
    WaxingCrescent,
    FullMoon,
    WaningCrescent
}

```

Скрипт AudioManager

```

public class AudioManager : MonoBehaviour
{
    public static AudioManager instance;
    [System.Serializable]
    public class SceneMusic
    {
        public string sceneName;
        public AudioClip backgroundMusic;
        public float volumeAudio;
    }
    public SceneMusic[] sceneMusics;
    private AudioSource audioSource;

    private bool isOnMusic = true;

    private void Awake()
    {
        if (instance == null)
        {
            instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
        }
        audioSource = gameObject.AddComponent<AudioSource>();
        audioSource.loop = true;
        audioSource.playOnAwake = false;
        audioSource.volume = 0.5f;
    }

    public void PlayMusicForScene(string sceneName)
    {
        foreach (var sceneMusic in sceneMusics)
        {
            if (sceneMusic.sceneName == sceneName)
            {
                if (audioSource.clip != sceneMusic.backgroundMusic)
                {
                    audioSource.Stop();
                    audioSource.clip = sceneMusic.backgroundMusic;
                    audioSource.volume = sceneMusic.volumeAudio;

                    if (isOnMusic)
                    {
                        audioSource.Play();
                    }
                }
                return;
            }
        }
        audioSource.Stop();
    }

    public void ToggleMusic()
    {
        isOnMusic = !isOnMusic;
        if (isOnMusic)
        {
            audioSource.Play();
        }
        else
        {
            audioSource.Pause();
        }
    }
}

```

```

    public bool IsMusicOn()
    {
        return isOnMusic;
    }
}

```

В.3 Скрипты сцен и другие

Скрипт CameraFollow

```

public class CameraFollow : MonoBehaviour
{
    public Transform target;
    public float lerpSpeed = 1.0f;
    private Vector3 offset;
    private Vector3 targetPos;

    private void Start()
    {
        target = GameObject.FindGameObjectWithTag("Player").transform;
        if (target == null) return;

        offset = transform.position - target.position;
    }
    private void Update()
    {
        if (target == null) return;

        targetPos = target.position + offset;

        transform.position = Vector3.Lerp(transform.position, targetPos, lerpSpeed *
Time.deltaTime);
    }
}

```

Скрипт SceneIntializer

```

public class SceneIntializer : MonoBehaviour
{
    public SceneType sceneType;
    private bool isFirstLoad = false;
    private void Start()
    {
        LevelSettings.instance.SetTypeScene(sceneType);

        if (isFirstLoad == false && sceneType == SceneType.FreeZone)
        {
            isFirstLoad = true;
            SaveManager.LoadGame();
        }
    }
}

```

Скрипт InteractionManager

```

public class InteractionManager : MonoBehaviour
{
    public static InteractionManager instance;
    private InteractionObject currentObj;
    private void Awake()
    {
        if (instance == null)
        {
            instance = this;
        }
    }
}

```

```

    }
    else
    {
        Destroy(gameObject);
    }
}

public void SetCurrentObj(InteractionObject obj)
{
    currentObj = obj;
}

public void ClearCurrentInteractable()
{
    currentObj = null;
}

public void OnInteractionButtonPressed()
{
    if (currentObj != null && currentObj.IsInRange())
    {
        currentObj.Interact();
    }
}
}

```

Скрипт InteractionObject

```

public class InteractionObject : MonoBehaviour
{
    public string nameInteraction;
    public SpriteRenderer spriteRendererObj;
    private Color normalColor;
    public Color highlightColor;

    private bool isInRange = false;

    public GameObject btnInteraction;
    public GameObject uiMessage;
    private void Start()
    {
        if (spriteRendererObj != null)
        {
            normalColor = spriteRendererObj.color;
        }
    }
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            isInRange = true;
            btnInteraction.SetActive(true);
            HighlightObj(true);
            InteractionManager.instance.SetCurrentObj(this);
            uiMessage.SetActive(true);
            Debug.Log("Коллайдеры столкновение");
        }
    }
    private void OnTriggerExit2D(Collider2D collision)
    {
        if (collision.CompareTag("Player"))
        {
            isInRange = false;
            btnInteraction.SetActive(false);
            HighlightObj(false);
        }
    }
}

```

```

        uiMessage.SetActive(false);
        InteractionManager.instance.ClearCurrentInteractable();
    }
}
private void HighlightObj(bool on)
{
    if (spriteRendererObj == null) return;
    spriteRendererObj.color = on ? Color.Lerp(normalColor, highlightColor, 0.5f)
: normalColor;
}
public virtual void Interact()
{
}
public bool IsInRange()
{
    return isInRange;
}
}

```

B.4 Менеджеры интерфейса

Скрипт UIManager

```

public class UIManager : MonoBehaviour
{
    public GameObject menuGame;
    public Sprite spOnAudio;
    public Sprite spOffAudio;
    public Image icAudio;
    private void Start()
    {
        if (SceneManager.GetActiveScene().name != "StartScene" &&
menuGame.activeSelf)
        {
            menuGame.SetActive(false);
        }
    }
    public void StartGame()
    {
        SceneManager.LoadScene("Village");
    }
    public void EducationOpen()
    {
        SceneManager.LoadScene("EducationScene");
    }
    public void OpenMenu()
    {
        menuGame.SetActive(true);
        if (AudioManager.instance != null)
        {
            UpdateAudioIcon(AudioManager.instance.IsMusicOn());
        }
        else
        {
            Debug.LogError("AudioManager instance is null! Cannot update audio
icon.");
        }
    }
    public void ExitGame()
    {
        SaveManager.SaveGame();
    }
}

```



```

        Application.Quit();
    }
    public void CloseMenu()
    {
        menuGame.SetActive(false);
    }
    public void AudioControl()
    {
        AudioManager.instance.ToggleMusic();
        UpdateAudioIcon(AudioManager.instance.IsMusicOn());
    }
    private void UpdateAudioIcon(bool isMusicOn)
    {
        if (icAudio == null)
        {
            Debug.LogError("icAudio is not assigned in UIManager!");
            return;
        }
        if (spOnAudio == null || spOffAudio == null)
        {
            Debug.LogError("spOnAudio or spOffAudio is not assigned in UIManager!");
            return;
        }
        icAudio.sprite = isMusicOn ? spOnAudio : spOffAudio;
    }
}

```

Скрипт MenuStatue

```

public class MenuStatue : MonoBehaviour
{
    public static MenuStatue instance;

    public GameObject menu;
    public TMP_Text moonShardInfo;
    public TMP_Text moonPhaseInfo;
    public TMP_Text aspectMenesInfo;
    public Image iconPhase;
    public Sprite iconPhaseWaxing;
    public Sprite iconPhaseFull;
    public Sprite iconPhaseWaning;
    void Start()
    {
        if (menu != null && menu.activeSelf)
        {
            menu.SetActive(false);
        }
    }
    private void Update()
    {
        LoadInfo();
    }
    public void CloseMenu()
    {
        menu.SetActive(false);
    }
    public void LoadInfo()
    {
        moonShardInfo.text = MenesManager.instance.GetMoonshard().ToString();
        AspectMenes curentAspectMenes = PlayerMenes.instance.GetCurrentAspect();
        switch (curentAspectMenes)
        {
            case (AspectMenes.Maid):
                moonPhaseInfo.text = "Возр.";
                aspectMenesInfo.text = "Дева";
        }
    }
}

```

```

        iconPhase.sprite = iconPhaseWaxing;
        break;
    case (AspectMenes.Mother):
        moonPhaseInfo.text = "Полн.";
        aspectMenesInfo.text = "Матерь";
        iconPhase.sprite = iconPhaseFull;
        break;
    case (AspectMenes.Crone):
        moonPhaseInfo.text = "Убыв.";
        aspectMenesInfo.text = "Старуха";
        iconPhase.sprite = iconPhaseWaning;
        break;
    }
}
}

```

Скрипт MenuExit

```

public class MenuExit : MonoBehaviour
{
    public GameObject menu;
    public string nameSceneGame;
    public void ViewMenu()
    {
        if (menu != null)
        {
            menu.SetActive(true);
        }
    }

    public void ExitLevel(bool isAgree)
    {
        if (isAgree && nameSceneGame != null)
        {
            Debug.Log("Метод отработал - ДА");
            SceneManager.instance.TransitionToScene(nameSceneGame);
        }
        else if (isAgree == false)
        {
            Debug.Log("Метод отработал - НЕТ");
            menu.SetActive(false);
        }
    }
}

```

Скрипт EducationManager

```

public class EducationManager : MonoBehaviour
{
    [Header("UI References")]
    public GameObject educationPanel;
    public GameObject choicePanel;
    public Image bgImage;
    public TMP_Text stepText;
    public TMP_Text titleText;
    public Button nextButton;
    public Button closeButton;
    [Header("Data")]
    public EducationTopic[] topics;
    private EducationTopic currentTopic;
    private int currentStepIndex = 0;

    private void Awake()
    {
        nextButton.onClick.AddListener(NextStep);
        closeButton.onClick.AddListener(ClosePanel);
    }
}

```

```

    }
    public void OpenTopic(int topicIndex)
    {
        if (topicIndex < 0 || topicIndex >= topics.Length)
        {
            Debug.LogError($"Topic index {topicIndex} out of range!");
            return;
        }
        currentTopic = topics[topicIndex];
        currentStepIndex = 0;
        bgImage.sprite = currentTopic.background;
        titleText.text = currentTopic.title;
        UpdateStepText();

        choicePanel.SetActive(false);
        educationPanel.SetActive(true);
    }
    private void NextStep()
    {
        if (currentTopic == null) return;

        currentStepIndex++;

        if (currentStepIndex >= currentTopic.steps.Length)
        {
        }
        else
        {
            UpdateStepText();
        }
    }
    private void UpdateStepText()
    {
        if (currentTopic != null && currentStepIndex < currentTopic.steps.Length)
        {
            stepText.text = currentTopic.steps[currentStepIndex];
        }
    }
    public void ClosePanel()
    {
        educationPanel.SetActive(false);
        choicePanel.SetActive(true);
        currentTopic = null;
        currentStepIndex = 0;
    }
    public void ExitEducation()
    {
        SceneManager.LoadScene("StartScene");
    }
}
[System.Serializable]
public class EducationTopic
{
    public string title;
    public Sprite background;
    public string[] steps;
}

```

B.5 Скрипты предметов, магазина, инвентаря

Скрипт Item

```

[CreateAssetMenu(fileName = "New Item", menuName = "Inventory/Item")]
public class Item : ScriptableObject
{

```

```

public string IdItem;
public string Name = "";
public string Description = "";
public Sprite Icon = null;
public int ratioShards = 1;
public int price = 1;
}

```

Скрипт ItemRegistry

```

public static class ItemRegistry
{
    private static List<Item> allItemGame = new List<Item>();

    [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
    private static void Initialize()
    {
        Item[] items = Resources.LoadAll<Item>("ItemCollection");
        allItemGame.AddRange(items);
        Debug.Log($"Loaded {allItemGame.Count} количество предметов
зарегистрированно.");
    }
    public static Item GetItemByID(string id)
    {
        foreach (var item in allItemGame)
        {
            if (item.IdItem == id) return item;
        }
        return null;
    }
}

```

Скрипт Inventory

```

public class Inventory : MonoBehaviour
{
    public static Inventory instance;
    public Transform SlotsParent;
    public InventorySlot[] inventorySlots = new InventorySlot[12];

    private void Awake()
    {
        if (instance == null)
        {
            instance = this;
        }
        else { Destroy(gameObject); }
    }
    public void PutInEmptySlot(Item item)
    {
        for (int i = 0; i < inventorySlots.Length; i++)
        {
            if (inventorySlots[i].itemInSlot == null)
            {
                inventorySlots[i].PutInSlot(item);
                return;
            }
        }
    }
    public int GetTotalBonus()
    {
        int totalBonus = 0;
        foreach (InventorySlot slot in inventorySlots)
        {
            if (slot.itemInSlot != null)
            {

```

```

        totalBonus += slot.itemInSlot.ratioShards;
    }
}
return totalBonus;
}
public string[] GetInventoryData()
{
    string[] data = new string[inventorySlots.Length];
    for (int i = 0; i < inventorySlots.Length; i++)
    {
        data[i] = inventorySlots[i].itemInSlot != null ?
inventorySlots[i].itemInSlot.IdItem : null;
    }
    return data;
}
public void ApplyInventoryData(string[] data)
{
    for (int i = 0; i < inventorySlots.Length && i < data.Length; i++)
    {
        if (!string.IsNullOrEmpty(data[i]))
        {
            Item item = ItemRegistry.GetItemByID(data[i]);

            if (item != null)
            {
                inventorySlots[i].PutInSlot(item);
            }
            else
            {
                Debug.LogWarning($"Item with ID '{data[i]}' not found in the
registry!");
                inventorySlots[i].ClearSlot();
            }
        }
        else
        {
            inventorySlots[i].ClearSlot();
        }
    }
}
}

```

Скрипт InventorySlot

```

public class InventorySlot : MonoBehaviour
{
    public Image icon;
    public Button buttonSlot;
    public Item itemInSlot;
    public void PutInSlot(Item item)
    {
        icon.sprite = item.Icon;
        icon.color = Color.white;
        itemInSlot = item;
    }
    public void SlotClicked()
    {
        if (itemInSlot != null)
        {
            ItemInfo.instance.Open(itemInSlot);
        }
        else { Debug.Log("Предмет не выбран!"); }
    }
    public void ClearSlot()
    {
    }
}

```

```

        icon.sprite = null;
        icon.color = new Color(255, 255, 255, 61);
        itemInSlot = null;
    }
}

```

Скрипт ItemInfo

```

public class ItemInfo : MonoBehaviour
{
    public static ItemInfo instance;
    private void Awake()
    {
        if (instance == null)
        {
            instance = this;
        }
        else { Destroy(gameObject); }
    }
    public GameObject InfoPanel;
    public TMP_Text Title;
    public TMP_Text DescriptionCountBonus;
    public TMP_Text DescriptionItem;
    public void ShowInfo(Item item)
    {
        Title.text = item.Name;
        DescriptionCountBonus.text = item.ratioShards.ToString();
        DescriptionItem.text = item.Description;
    }
    public void Open(Item item)
    {
        ShowInfo(item);
        InfoPanel.SetActive(true);
    }
    public void Close()
    {
        InfoPanel.SetActive(false);
    }
}

```

Скрипт StoreManager

```

public class StoreManager : MonoBehaviour
{
    public static StoreManager instance;

    [Header("UI References")]
    public TMP_Text shardsText;
    public Button buyButton;
    public Button closeButton;

    public Transform SlotsParent;
    public GameObject menuStore;

    private StoreSlot selectedSlot;

    [Header("Data")]
    public StoreSlot[] itemSlots = new StoreSlot[12];

    private int currentShards;
    void Awake()
    {
        if (instance == null)
        {
            instance = this;
        }
    }
}

```

```

else { Destroy(gameObject); }

closeButton.onClick.RemoveAllListeners();
closeButton.onClick.AddListener(CloseStore);

for (int i = 0; i < itemSlots.Length; i++)
{
    itemSlots[i] = SlotsParent.GetChild(i).GetComponent<StoreSlot>();
    itemSlots[i].Initialize(this);
}
SelectItem(itemSlots[0]);
buyButton.onClick.AddListener(TryBuyItem);
buyButton.interactable = false;
UpdateShardsDisplay();
}
private void Start()
{
    if (menuStore != null && menuStore.activeSelf)
    {
        menuStore.SetActive(false);
    }
}
private void OnEnable()
{
    currentShards = MenesManager.instance.GetMoonshard();
    UpdateShardsDisplay();
}
public void SelectItem(StoreSlot slot)
{
    if (slot.lockedOverlay.activeSelf) return;

    selectedSlot = slot;
    buyButton.interactable = true;
}
public void TryBuyItem()
{
    if (selectedSlot == null) return;

    int cost = selectedSlot.item.price;

    if (currentShards >= cost)
    {
        selectedSlot.SetAsPurchased();
        currentShards -= cost;
        MenesManager.instance.GiveMoonshard(cost);
        UpdateShardsDisplay();
        buyButton.interactable = false;
        Inventory.instance.PutInEmptySlot(selectedSlot.item);
        Debug.Log("Куплено!");
        return;
    }
    else
    {
        Debug.Log("Недостаточно лунных осколков!");
        return;
    }
}
public bool[] GetStoreData()
{
    bool[] data = new bool[itemSlots.Length];

    for (int i = 0; i < itemSlots.Length; i++)
    {
        data[i] = itemSlots[i].lockedOverlay.activeSelf;
    }
}

```

```

        return data;
    }

    public void ApplyStoreData(bool[] data)
    {
        for (int i = 0; i < data.Length && i < itemSlots.Length; i++)
        {
            if (data[i])
            {
                itemSlots[i].SetAsPurchased();
            }
        }
    }

    public void CloseStore()
    {
        menuStore.SetActive(false);
        buyButton.interactable = false;
        selectedSlot = itemSlots[0];
    }

    private void UpdateShardsDisplay()
    {
        shardsText.text = currentShards.ToString();
    }
}

```

Скрипт StoreSlot

```

public class StoreSlot : MonoBehaviour
{
    public Item item;
    private Image iconItem;
    private TMP_Text itemNameText;
    private TMP_Text itemCountBonusText;
    private TMP_Text itemDescText;
    private TMP_Text itemCountPrice;
    public Button slotButton;
    public GameObject lockedOverlay;
    public Transform infoPanel;
    public StoreManager shopManager;
    public void Initialize(StoreManager manager)
    {
        shopManager = manager;
        slotButton = GetComponent<Button>();
        slotButton.onClick.AddListener(OnClickSlot);
        if (item != null)
        {
            iconItem = transform.GetChild(0).GetComponent<Image>();
            iconItem.sprite = item.Icon;
            iconItem.color = Color.white;
            itemNameText = infoPanel.transform.GetChild(0).GetComponent<TMP_Text>();
            itemCountBonusText =
infoPanel.transform.GetChild(2).GetComponent<TMP_Text>();
            itemDescText = infoPanel.transform.GetChild(3).GetComponent<TMP_Text>();
            itemCountPrice =
infoPanel.transform.GetChild(5).GetComponent<TMP_Text>();
            UpdateUI();
        }
    }

    public void OnClickSlot()
    {
        if (lockedOverlay.activeSelf) return;
        shopManager.SelectItem(this);
        UpdateUI();
    }
}

```



```

    }

    public void UpdateUI()
    {
        if (item == null) return;
        itemNameText.text = item.Name;
        itemDescText.text = item.Description;
        itemCountBonusText.text = item.ratioShards.ToString();
        itemCountPrice.text = item.price.ToString();
    }

    public void SetAsPurchased()
    {
        lockedOverlay.SetActive(true);
        slotButton.interactable = false;
    }
}

```

В.6 Скрипты сохранения

Скрипт GameSaveData

```

[System.Serializable]
public class GameSaveData
{
    public int moonshards;
    public string currentAspect;
    public string currentMoonPhase;
    public float phaseTimer;
    public bool[] storeSlots;
    public string[] itemIDs;
    public int shardBonus;
}

```

Скрипт SaveManager

```

public static class SaveManager
{
    private static string SavePath => Path.Combine(Application.persistentDataPath,
"save.json");
    public static void SaveGame()
    {
        GameSaveData data;
        if (File.Exists(SavePath))
        {
            try
            {
                string jsonRead = File.ReadAllText(SavePath);
                data = JsonUtility.FromJson<GameSaveData>(jsonRead);
                if (data == null)
                {
                    data = new GameSaveData();
                }
            }
            catch
            {
                data = new GameSaveData();
            }
        }
        else
        {
            data = new GameSaveData();
        }
        data.moonshards = MenesManager.instance.GetMoonshard();
        data.currentAspect = PlayerMenes.instance.GetCurrentAspect().ToString();
    }
}

```

```

        data.currentMoonPhase =
MoonPhaseManager.instance.GetCurrentPhase().ToString();
        data.phaseTimer = MoonPhaseManager.instance.GetTimer();
        if (SceneManager.GetActiveScene().name == "Village")
        {
            data.storeSlots = StoreManager.instance.GetStoreData();
            data.itemIDs = Inventory.instance.GetInventoryData();
            data.shardBonus = Inventory.instance.GetTotalBonus();
            Debug.Log("Инвентарь и магазин сохранены");
        }
        string json = JsonUtility.ToJson(data, true);
        File.WriteAllText(SavePath, json);

        Debug.Log($"Игра сохранена в {SavePath}");
    }
    public static void LoadGame()
    {
        try
        {
            if (File.Exists(SavePath))
            {
                string json = File.ReadAllText(SavePath);
                GameSaveData data = JsonUtility.FromJson<GameSaveData>(json);
                if (MenesManager.instance == null)
                {
                    Debug.Log("Менес не найден");
                }
                if (MoonPhaseManager.instance == null)
                {
                    Debug.Log("MoonPhas не найден");
                }
                MenesManager.instance.SetMoonshard(data.moonshards);
                MenesManager.instance.SetNewAspect((AspectMenes)Enum.Parse(typeof(AspectMenes),
                data.currentAspect));
                MoonPhaseManager.instance.SetPhase((MoonPhase)Enum.Parse(typeof(MoonPhase),
                data.currentMoonPhase), data.phaseTimer);

                if (SceneManager.GetActiveScene().name == "Village")
                {
                    StoreManager.instance.ApplyStoreData(data.storeSlots);
                    Inventory.instance.ApplyInventoryData(data.itemIDs);
                }
            }
            else
            {
                SaveGame();
            }
        }
        catch (Exception e)
        {
            Debug.LogError($"Ошибка при загрузке сохранения: {e.Message}");
        }
    }
    public static bool SaveExists()
    {
        return File.Exists(SavePath);
    }
    public static void DeleteSave()
    {
        if (File.Exists(SavePath))
        {
            File.Delete(SavePath);
            Debug.Log("Сохранение удалено");
        }
    }
}

```

Г.1 Скрипты мини-игры «Зеркальца»

Скрипт Mirror

```

public class Mirror : MonoBehaviour
{
    public float angleStep = 45f;
    private AudioSource audioMirrorSFX;
    private Camera mainCamera;
    void Start()
    {
        mainCamera = Camera.main;
        audioMirrorSFX = GetComponent<AudioSource>();
    }
    void Update()
    {
        if (Input.touchCount > 0)
        {
            Touch touch = Input.GetTouch(0);
            if (touch.phase == TouchPhase.Began)
            {
                Vector3 touchPosition = mainCamera.ScreenToWorldPoint(new
Vector3(touch.position.x, touch.position.y, 10));
                RaycastHit2D hit = Physics2D.Raycast(touchPosition, Vector2.zero);
                if (hit.collider != null && hit.collider.gameObject ==
this.gameObject)
                {
                    RotateMirror();
                    audioMirrorSFX.Play();
                }
            }
        }
    }
    private void RotateMirror()
    {
        transform.Rotate(0, 0, angleStep);
    }
}

```

Скрипт RaySystem

```

public class RaySystem : MonoBehaviour
{
    public Transform startPoint;
    public Transform endPoint;
    public LineRenderer lineRenderer;
    public LayerMask mirrorLayer;
    private bool isCompleted = false;
    public int maxReflections = 3;
    public float lenthRay = 50f;
    public TMP_Text txtCountReflection;
    public TMP_Text txtLenthRay;
    private int shardsCountLevel = 10;
    void Update()
    {
        if (!isCompleted)
        {
            DrawRay();
            UpdateInfoRay();
        }
    }
}

```

```

private void DrawRay()
{
    lineRenderer.positionCount = 0;
    Vector2 currentDirection = Vector2.up;
    Vector2 currentPosition = startPoint.position;
    lineRenderer.positionCount = 1;
    lineRenderer.SetPosition(0, currentPosition);
    for (int i = 0; i < maxReflections; i++)
    {
        RaycastHit2D hit = Physics2D.Raycast(currentPosition, currentDirection,
lenthRay, mirrorLayer);

        if (hit.collider != null)
        {
            currentPosition = hit.point;
            lineRenderer.positionCount++;
            lineRenderer.SetPosition(lineRenderer.positionCount - 1,
currentPosition);
            currentDirection = Vector2.Reflect(currentDirection, hit.normal);
        }
        else
        {
            currentPosition += currentDirection * lenthRay;
            lineRenderer.positionCount++;
            lineRenderer.SetPosition(lineRenderer.positionCount - 1,
currentPosition);
            break;
        }
    }
    if (Vector2.Distance(currentPosition, endPoint.position) < 1f)
    {
        isCompleted = true;
        Debug.Log("Луч достиг точки приема!");
        OnRayReachedTarget();
        return;
    }
}

private void UpdateInfoRay()
{
    txtCountReflection.text = maxReflections.ToString();
    txtLenthRay.text = lenthRay.ToString();
}

private void OnRayReachedTarget()
{
    RayPointReceptionAnim anim = endPoint.GetComponent<RayPointReceptionAnim>();
    if (anim != null)
    {
        anim.StartAnimation();
    }
    EndMiniGame.instance.EndMinigame(shardsCountLevel);
}
}

```

Скрипт RayPointReceptionAnim

```

public class RayPointReceptionAnim : MonoBehaviour
{
    private SpriteRenderer spriteRenderer;
    private bool isAnimating = false;
    private void Start()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        spriteRenderer.color = Color.white;
    }
    public void StartAnimation()

```

```

{
    isAnimating = true;
    StartCoroutine(AnimatePoint());
}
private IEnumerator AnimatePoint()
{
    while (isAnimating)
    {
        float intensity = Mathf.PingPong(Time.time, 1f);
        spriteRenderer.color = Color.Lerp(Color.white, Color.blue, intensity);
        yield return null;
    }
}
}

```

Г.2 Скрипты мини-игры «Камушки-Кумушки»

Скрипт MatchManager

```

public class MatchManager : MonoBehaviour
{
    private int countPoint;
    public TMP_Text countPointText;
    private int countTarget = 0;
    public TMP_Text countTargetText;
    public GameObject[,] gemsGrid = new GameObject[8, 5];
    public GameObject[] gemsArray;
    public int gemsMoveCounter = 0;
    public bool canMove = false;
    public bool canCheck = false;
    public bool canRefill = false;
    public bool noMatches = true;
    public bool isPurpose = false;
    private void Start()
    {
        CountingTarget();
    }
    private void Update()
    {
        if (canCheck && gemsMoveCounter <= 0)
        {
            canCheck = false;
            gemsMoveCounter = 0;
            GetComponent<GridChecker>().CheckForMatches();
        }
        if (canRefill && gemsMoveCounter <= 0)
        {
            canRefill = false;
            gemsMoveCounter = 0;
            GetComponent<RefillGrid>().Refill();
        }
        if (!canRefill && noMatches && !canCheck && gemsMoveCounter <= 0)
        {
            gemsMoveCounter = 0;
            canMove = true;
        }
    }
    private void CountingTarget()
    {
        AspectMenes currentAspect = PlayerMenes.instance.GetCurrentAspect();
        int target = UnityEngine.Random.Range(3, 334);
        double coeff = 0;
        switch (currentAspect)
        {
            case AspectMenes.Maid:

```

```

        coeff = 1.3;
        break;
    case AspectMenes.Mother:
        coeff = 1;
        break;
    case AspectMenes.Crone:
        coeff = 0.3;
        break;
    }
    countTarget = (int)Math.Round((target + (target * coeff)));
    countTargetText.text = countTarget.ToString();
}
public void CountingPoint(int countMatch)
{
    if (countMatch > 0)
    {
        countPoint = countPoint + (10 * countMatch);
    }
    countPointText.text = countPoint.ToString();
    CheckTargetPoint();
}
private void CheckTargetPoint()
{
    if (countPoint >= countTarget)
    {
        isPurpose = true;
        int giveShard = Mathf.RoundToInt(Mathf.Sqrt(countTarget) * 3);
        EndMiniGame.instance.EndMinigame(giveShard);
    }
}
}

```

Скрипт GemData

```

public class GemData : MonoBehaviour
{
    public int gridX, gridY;
    private CircleCollider2D colliderGem;
    private bool canMoveToPose = false;
    public float speed;
    public Vector3 endPos;
    private MatchManager manager;
    private void Awake()
    {
        manager =
        GameObject.FindGameObjectWithTag("Manager3Match").GetComponent<MatchManager>();
        colliderGem = GetComponent<CircleCollider2D>();
    }
    private void Update()
    {
        if (canMoveToPose)
        {
            Vector3 dir = (endPos - transform.position).normalized;
            transform.Translate(dir * speed * Time.deltaTime);
            if ((endPos - transform.position).sqrMagnitude < 0.05f)
            {
                transform.position = endPos;
                colliderGem.enabled = true;
                manager.gemsMoveCounter--;
                GetComponent<SpriteRenderer>().sortingOrder = 0;
                canMoveToPose = false;
            }
        }
    }
    public void Move()

```

```

    {
        endPos = new Vector3(gridX, gridY);
        colliderGem.enabled = false;
        manager.gemsMoveCounter--;
        GetComponent<SpriteRenderer>().sortingOrder = 10;
        canMoveToPose = true;
    }
}

```

Скрипт GridChecker

```

public class GridChecker : MonoBehaviour
{
    private List<List<GameObject>> matchedGems;
    private MatchManager manager;
    private GameObject[,] gemsGrid;
    public void CheckForMatches()
    {
        manager = GetComponent<MatchManager>();
        gemsGrid = manager.gemsGrid;
        matchedGems = new List<List<GameObject>>();
        GameObject firstGemToCheck = null;
        List<GameObject> tmpGems = new List<GameObject>();
        for (int y = 0; y < gemsGrid.GetLength(1); y++)
        {
            firstGemToCheck = null;
            tmpGems.Clear();
            for (int x = 0; x < gemsGrid.GetLength(0); x++)
            {
                GameObject current = gemsGrid[x, y];
                if (current == null)
                {
                    if (tmpGems.Count >= 3)
                        matchedGems.Add(new List<GameObject>(tmpGems));
                    tmpGems.Clear();
                    firstGemToCheck = null;
                    continue;
                }
                if (firstGemToCheck == null || current.tag != firstGemToCheck.tag)
                {
                    if (tmpGems.Count >= 3)
                        matchedGems.Add(new List<GameObject>(tmpGems));
                    tmpGems.Clear();
                    tmpGems.Add(current);
                    firstGemToCheck = current;
                }
                else
                {
                    tmpGems.Add(current);
                }
            }
            if (tmpGems.Count >= 3)
                matchedGems.Add(new List<GameObject>(tmpGems));
        }
        for (int x = 0; x < gemsGrid.GetLength(0); x++)
        {
            firstGemToCheck = null;
            tmpGems.Clear();
            for (int y = 0; y < gemsGrid.GetLength(1); y++)
            {
                GameObject current = gemsGrid[x, y];
                if (current == null)
                {
                    if (tmpGems.Count >= 3)
                        matchedGems.Add(new List<GameObject>(tmpGems));
                }
            }
        }
    }
}

```

```

        tmpGems.Clear();
        firstGemToCheck = null;
        continue;
    }
    if (firstGemToCheck == null || current.tag != firstGemToCheck.tag)
    {
        if (tmpGems.Count >= 3)
            matchedGems.Add(new List<GameObject>(tmpGems));
        tmpGems.Clear();
        tmpGems.Add(current);
        firstGemToCheck = current;
    }
    else
    {
        tmpGems.Add(current);
    }
}
if (tmpGems.Count >= 3)
    matchedGems.Add(new List<GameObject>(tmpGems));
}
int totalMatchedGems = 0;
foreach (List<GameObject> group in matchedGems)
{
    totalMatchedGems += group.Count;
    foreach (GameObject gem in group)
    {
        if (gem != null)
        {
            GemData data = gem.GetComponent<GemData>();
            manager.gemsGrid[data.gridX, data.gridY] = null;
            Destroy(gem);
        }
    }
}
manager.CountingPoint(totalMatchedGems);
matchedGems.Clear();
if (manager.isPurpose == false)
{
    if (totalMatchedGems > 0)
    {
        manager.noMatches = false;
        manager.canRefill = true;
        manager.canCheck = false;
    }
    else
    {
        manager.noMatches = true;
        manager.canRefill = false;
        manager.canCheck = false;
    }
}
}
}
}

```

Скрипт MoveGem

```

public class MoveGem : MonoBehaviour
{
    public float swipeThreshold = 0.5f;
    private string direction;
    private GameObject selectedGem;
    private Vector2 startSwap;
    private Vector2 endSwap;
    private MatchManager manager;
    private void Start()
    {

```



```

{
    manager = GetComponent<MatchManager>();
}
private void Update()
{
    if (Input.touchCount > 0 && manager.canMove)
    {
        Touch touch = Input.GetTouch(0);
        switch (touch.phase)
        {
            case TouchPhase.Began:
                Vector3 touchPos =
Camera.main.ScreenToWorldPoint(touch.position);
                startSwap = touchPos;
                RaycastHit2D hit = Physics2D.Raycast(startSwap, Vector2.zero);
                if (hit.collider != null)
                {
                    selectedGem = hit.collider.gameObject;
                }
                break;
            case TouchPhase.Ended:
                if (selectedGem != null)
                {
                    Vector3 endTouchPos =
Camera.main.ScreenToWorldPoint(touch.position);
                    endSwap = endTouchPos;
                    Vector2 delta = endSwap - startSwap;
                    direction = "";
                    if (delta.magnitude > swipeThreshold)
                    {
                        if (Mathf.Abs(delta.x) >= Mathf.Abs(delta.y))
                            direction = delta.x > 0 ? "right" : "left";
                        else
                            direction = delta.y > 0 ? "up" : "down";
                    }
                    if (direction != "")
                    {
                        SwitchGems(direction);
                    }
                    selectedGem = null;
                }
                break;
        }
    }
}

void SwitchGems(string direction)
{
    if (selectedGem == null)
    {
        Debug.LogWarning("Selected gem is null!");
        return;
    }
    switch (direction)
    {
        case "right": GemSwap(Vector2Int.right); break;
        case "left": GemSwap(Vector2Int.left); break;
        case "up": GemSwap(Vector2Int.up); break;
        case "down": GemSwap(Vector2Int.down); break;
    }
}

void GemSwap(Vector2Int dir)
{
    int x1 = selectedGem.GetComponent<GemData>().gridX;
    int y1 = selectedGem.GetComponent<GemData>().gridY;
    int x2 = x1 + dir.x;

```

```

        int y2 = y1 + dir.y;
        if (x2 < 0 || x2 >= manager.gemsGrid.GetLength(0) || y2 < 0 || y2 >=
manager.gemsGrid.GetLength(1))
            return;
        GameObject gem1 = manager.gemsGrid[x1, y1];
        GameObject gem2 = manager.gemsGrid[x2, y2];
        var data1 = gem1.GetComponent<GemData>();
        var data2 = gem2.GetComponent<GemData>();
        data1.gridX = x2;
        data1.gridY = y2;
        data2.gridX = x1;
        data2.gridY = y1;
        data1.endPos = new Vector3(x2, y2);
        data2.endPos = new Vector3(x1, y1);
        manager.gemsGrid[x1, y1] = gem2;
        manager.gemsGrid[x2, y2] = gem1;
        data1.Move();
        data2.Move();
        manager.canMove = false;
        manager.canCheck = true;
    }
}

```

Скрипт RefillGrid

```

public class RefillGrid : MonoBehaviour
{
    private MatchManager manager;
    private GameObject[,] gemsGrid;
    private GameObject[] gemsArray;
    public Transform parentObject;
    private void Start()
    {
        manager = GetComponent<MatchManager>();
        gemsArray = manager.gemsArray;
        gemsGrid = manager.gemsGrid;
    }
    public void Refill()
    {
        StartCoroutine(DoRefillSequence());
        manager.canRefill = false;
    }
    IEnumerator DoRefillSequence()
    {
        yield return StartCoroutine(SlowFall());
        yield return StartCoroutine(SlowSpawn());
        yield return new WaitForSeconds(1f);
        manager.canCheck = true;
    }
    IEnumerator SlowFall()
    {
        for (int x = 0; x < gemsGrid.GetLength(0); x++)
        {
            for (int y = 0; y < gemsGrid.GetLength(1) - 1; y++)
            {
                if (gemsGrid[x, y] == null)
                {
                    for (int aboveY = y + 1; aboveY < gemsGrid.GetLength(1);
aboveY++)
                    {
                        if (gemsGrid[x, aboveY] != null)
                        {
                            GameObject gem = gemsGrid[x, aboveY];
                            gemsGrid[x, y] = gem;
                            gemsGrid[x, aboveY] = null;

```

```

        GemData data = gem.GetComponent<GemData>();
        data.gridY = y;
        data.Move();
        yield return new WaitForSeconds(0.1f);
        break;
    }
}
}
}
IEnumerator SlowSpawn()
{
    for (int x = 0; x < gemsGrid.GetLength(0); x++)
    {
        for (int y = 0; y < gemsGrid.GetLength(1); y++)
        {
            if (gemsGrid[x, y] == null)
            {
                GameObject newGem = Instantiate(
                    gemsArray[Random.Range(0, gemsArray.Length)],
                    new Vector2(x, gemsGrid.GetLength(1) + 1),
                    Quaternion.identity,
                    parentObject
                );
                gemsGrid[x, y] = newGem;
                GemData data = newGem.GetComponent<GemData>();
                data.gridX = x;
                data.gridY = y;
                data.Move();
                yield return new WaitForSeconds(0.2f);
            }
        }
    }
}
}
}
}
}

```

Скрипт SpawnController

```

public class SpawnController : MonoBehaviour
{
    private MatchManager manager;
    public Transform parentObject;
    public float spacing = 1.5f;
    void Start()
    {
        manager = GetComponent<MatchManager>();
        FillTheGrid();
    }
    public void FillTheGrid()
    {
        for (int i = 0; i < 8; i++)
        {
            for (int j = 0; j < 5; j++)
            {
                Vector2 pos = new Vector2(i, j);
                List<GameObject> candidateTypes = new
List<GameObject>(manager.gemsArray);
                if (i >= 1)
                    candidateTypes.RemoveAll(gem => gem.tag == manager.gemsGrid[i -
1, j].tag);
                if (j >= 1)
                    candidateTypes.RemoveAll(gem => gem.tag == manager.gemsGrid[i, j
- 1].tag);
            }
        }
    }
}

```

```

        GameObject gem = Instantiate(candidateTypes[Random.Range(0,
candidateTypes.Count)]),
            pos,
            Quaternion.identity,
            parentObject);
        GemData gemData = gem.GetComponent<GemData>();
        gemData.gridX = i;
        gemData.gridY = j;
        manager.gemsGrid[i, j] = gem;
    }
}
manager.canMove = true;
}
}

```

Г.3 Скрипт EndMiniGame

```

public class EndMiniGame : MonoBehaviour
{
    public static EndMiniGame instance;
    public TMP_Text txtShardCount;
    public GameObject menuWin;
    public string nameSceneExitGame;
    private Animator animator;
    private void Start()
    {
        if (instance == null)
        {
            instance = this;
        }
        else { Destroy(gameObject); }
        animator = menuWin.GetComponent<Animator>();
    }
    public void EndMinigame(int shard)
    {
        if (menuWin != null)
        {
            menuWin.SetActive(true);
            animator.SetTrigger("OpenMenu");
            int shardBonus = LoadTotalBonusFromSave();
            int totalShards = shard + shardBonus;
            MenesManager.instance.TakeMoonshard(totalShards);
            txtShardCount.text = totalShards.ToString();
        }
    }
    public void ButtonContinue()
    {
        SaveManager.SaveGame();
        if (nameSceneExitGame == null)
        {
            SceneManager.LoadScene("Village");
        }
        else
        {
            SceneManager.LoadScene(nameSceneExitGame);
        }
    }
    private int LoadTotalBonusFromSave()
    {
        string path = Path.Combine(Application.persistentDataPath, "saves.json");
        if (File.Exists(path))
        {
            try
            {
                string json = File.ReadAllText(path);
            }
        }
    }
}

```

```
        GameSaveData data = JsonUtility.FromJson<GameSaveData>(json);
        return data?.shardBonus ?? 0;
    }
    catch (System.Exception e)
    {
        Debug.LogError($"Ошибка при загрузке totalBonus: {e.Message}");
    }
}
return 0;
}
```

Таблица Д.1 — Функционально тестирование игры

Тестируемый параметр	Ожидаемое значение	Фактическое значение
Механика перемещения персонажа		
Реакция на управление через джойстик	Персонаж движется в направлении, указанном джойстиком	Персонаж двигается в соответствии с управлением
Скорость перемещения в аспекте «Дева»	Скорость перемещения равна 5 единиц	При аспекте «Дева» скорость равна 5
Скорость перемещения в аспекте «Мать»	Скорость перемещения равна 4 единицы	При аспекте «Мать» скорость равна 4
Скорость перемещения в аспекте «Старуха»	Скорость перемещения равна 3 единицы	При аспекте «Старуха» скорость равна 3
Корректное движение по диагонали	Персонаж движется с правильной скоростью по диагонали (не быстрее, чем по осям)	Движение по диагонали корректное
Остановка персонажа при отпускании джойстика	Персонаж останавливается мгновенно после отпускания джойстика	Персонаж останавливается моментально при отпускании джойстика
Отсутствие движения при значении джойстика ниже мертвой зоны	Персонаж не двигается при небольших отклонениях джойстика (менее 0.1)	Персонаж не двигается при малом отклонении джойстика
Корректное движение вдоль границ локации	Персонаж не выходит за пределы локации и останавливается у границ	Персонаж не выходит за границы, не застревает в них
Обработка столкновений с препятствиями	Персонаж не проходит сквозь препятствия и корректно взаимодействует с ними	Персонаж обходит препятствия, не застревает
Плавность движения персонажа	Движение персонажа плавное, без рывков и подергиваний	Персонаж плавно передвигается во время всего времени
Корректное обновление аспекта при смене фазы луны	Скорость персонажа изменяется сразу после смены фазы луны	При смене фазы сразу происходит смена скорости
Сохранение скорости при переходе между сценами	Скорость персонажа сохраняется при переходе между сценами	Значение скорости сохраняется между сценами
Корректная работа на разных разрешениях экрана	Управление джойстиком корректно работает на всех поддерживаемых разрешениях	Джойстик корректно работает на разных экранах
Головоломка «Зеркальца»		
Возможность поворота зеркал при касании экрана	Зеркало поворачивается на 45° в заданном направлении	Зеркало поворачивается ровно на 45°
Отражение лунного луча от зеркала	Луч отражается под правильным углом	Луч отражается от зеркала в правильном направлении
Достижение лучом точки приема (алтаря)	При правильной настройке зеркал луч достигает алтаря	Луч достигает алтаря при необходимых настройках
Активация завершения уровня при достижении лучом точки приема	Точка приема начинает анимироваться	Точка анимируется
	Открывается панель завершения мини-игры	Открывается панель конца уровня
Ограничение количества отражений	Луч не отражается более заданного количества раз	Луч ограничивается заданным количеством отражений
Отображение количества зеркал	В UI отображается корректное количество возможных зеркал	На экране корректно отображаются количество возможных зеркал
Мини-игра «Камушки-Кумушки»		

Тестируемый параметр	Ожидаемое значение	Фактическое значение
Игровое поле корректно заполняется при запуске уровня	Поле заполняется случайными камнями, начальные совпадения отсутствуют или удаляются автоматически	Поле заполнено корректно, начальные совпадения обработаны
Перемещение камней касанием экрана	Камень перемещается в соседнюю ячейку только при свайпе, обмен происходит мгновенно	Камни перемещаются по свайпу без ошибок
Удаление совпадающих рядов (3+ камня)	При сборе 3 и более одинаковых камней они исчезают с поля	Совпадения удаляются корректно
Начисление очков за удаление камней	За каждый камень в удаленном ряду начисляется 10 очков, счетчик обновляется	Очки начисляются верно, UI обновляется
Перезаполнение поля после удаления	На место удаленных камней падают новые сверху, поле остается полным	Новые камни появляются и занимают пустые ячейки
Подсчет цели уровня с учетом аспекта	Целевое количество очков рассчитывается по формуле с коэффициентом текущего аспекта	Цель уровня соответствует текущему аспекту персонажа
Завершение мини-игры при достижении цели	После набора нужного количества очков открывается панель завершения	Панель завершения открывается вовремя
Система смены фаз Луны		
Интервал смены фаз	Смена фазы происходит каждые 15 минут реального времени	Фазы сменяются каждые 15 минут
Последовательность фаз	Новолуние → Полнолуние → Старая Луна → Новолуние и т. д.	Последовательность фаз сохранена
Обновление индикатора фазы на экране	Индикатор отображает текущую фазу луны	Индикатор отображает верную фазу
Сохранение состояния фазы при переходе между сценами	Фаза продолжает меняться независимо от текущей сцены	Фаза сохраняется между сценами
Изменение аспекта персонажа при смене фазы	Скорость персонажа меняется в соответствии с фазой	Аспект сменяется вместе с фазой корректно
Перемещение между локациями		
Переход из Деревни Ильмала в Святилище Неба	Успешный переход при взаимодействии с точкой перехода	Персонаж появляется в нужной точке
Переход из Деревни Ильмала в Храм первого света	Успешный переход при взаимодействии с точкой перехода	Персонаж появляется в нужной точке
Возврат из Святилища Неба в Деревню Ильмала	Успешный возврат к соответствующей точке входа в деревне	Персонаж появляется в нужной точке
Возврат из Храма первого света в Деревню Ильмала	Успешный возврат к соответствующей точке входа в деревне	Персонаж появляется в нужной точке
Интерактивные объекты		
Отображение кнопки взаимодействия при приближении	Кнопка взаимодействия появляется на экране	При подходе к объекту на экране появляется кнопка взаимодействия
Изменение спрайта объекта при приближении	Спрайт изменяет цвет/прозрачность	При подходе к объекту его спрайт меняется
Уникальные действия для разных типов объектов	Разные объекты выполняют разные действия при взаимодействии	Объект выполняет заданную функцию
	Дорожный знак: загружает новую сцену свободной зоны	Объект выполняет заданную функцию
	Алтарь: загружает сцену головоломки	Объект выполняет заданную функцию
	Статуя: открывает панель информации	Объект выполняет заданную функцию

Тестируемый параметр	Ожидаемое значение	Фактическое значение
Система наград		
Выдача Лунных осколков за успешное решение головоломки	За решенную головоломку выдается фиксированное значение в сумме с бонусом от предметов	Суммирование бонуса и значения происходит верно
Отображение количества собранных осколков в UI	Количество осколков отображается в соответствующем элементе интерфейса	На панели конца уровня отображаются количество начисленных осколков
Сохранение количества осколков при переходе между сценами	Количество осколков не сбрасывается при переходе между сценами	Количество осколков сохраняется при переходах
Обновление количества осколков при получении нового	UI обновляется мгновенно после получения нового осколка	После получения количество осколков отображается корректно
Лавка (Магазин)		
Блокировка предмета после покупки	Ячейка переходит в состояние «куплено», кнопка покупки становится неактивной	Предмет заблокирован, повторная покупка невозможна
Списание Лунных осколков при покупке	Количество ЛО уменьшается ровно на стоимость предмета	Баланс ЛО списан корректно
Сохранение состояния ячеек магазина	Купленные предметы остаются купленными после перезагрузки игры и переходов между сценами	Состояние магазина сохраняется всегда
Отображение информации о выбранном предмете	При выборе предмета в панели отображаются название, описание, цена и бонус	Информация выводится полностью и верно
Инвентарь		
Занесение предметов в ячейки инвентаря	Купленные предметы появляются в первом свободном слоте инвентаря	Предметы добавляются в правильные ячейки
Сохранение содержимого инвентаря	Список предметов и их позиции сохраняются между сессиями и сценами	Инвентарь восстанавливается полностью
Корректный вывод данных в панели информации	Всплывающее окно показывает актуальные характеристики хранящегося предмета	Данные в панели соответствуют предмету
Открытие и закрытие панели информации	Панель плавно появляется при нажатии и скрывается при нажатии вне зоны или на крестик	Анимация открытия/закрытия работает без сбоев